

## REVIEW

# A comprehensive review of convolutional neural networks: foundations, enhancements and applications

Md. Himel Reza · Md. Noman Biswas Sibly · Shaikh Golam Rabbani ·  
Shafayetul Huda Sadi · Md. Faysal Ahamed · Fariya Bintay Shafi · Rusab Sarmun ·  
Muhammad E. H. Chowdhury 

Received: 22 March 2025 / Accepted: 22 September 2025 / Published online: 18 February 2026  
© The Author(s) 2026

## Abstract

Convolutional Neural Networks (CNNs) have emerged as a cornerstone in the field of deep learning, demonstrating remarkable performance across various domains, including computer vision and natural language processing. Their widespread acceptance on both academic and industrial levels has spurred much research and development. This study provides a comprehensive overview of recent developments in convolutional neural network (CNN) architectures by analyzing their foundational concepts, structural enhancements, and various applications such as image classification, medical imaging, and autonomous systems. Starting with conventional CNN architectures and their fundamental elements, the paper uses a systematic approach to look into recent advances. Key architectural innovations, including advanced activation functions, novel pooling strategies, and optimized convolutional techniques, are discussed. The study also explores hybrid architectures that integrate CNNs with transformers and recurrent neural networks to enhance contextual and sequential learning. Advances in training processes, such as enhanced loss functions and regularization approaches, have been studied for enhancing model performance. The study highlights CNN advancements that improve accuracy, reduce computational costs, and enhance model generalization. It underscores the effectiveness of CNNs in critical domains. Findings reveal that improved feature extraction techniques and interpretability methods, including Gradient-weighted Class Activation Mapping (Grad-CAM) and Graph-CNNs, contribute significantly to CNN performance. Finally, the paper identifies open challenges and outlines potential research directions, providing insights into the future development of CNNs.

**Keywords** Convolutional neural networks (CNNs) · Deep learning · Medical imaging · Neural network optimization

## Abbreviations

|         |                            |
|---------|----------------------------|
| Adam    | Adaptive moment estimation |
| AI      | Artificial Intelligence    |
| BN      | Batch normalization        |
| CapsNet | Capsule Network            |



|          |                                                   |
|----------|---------------------------------------------------|
| CNN      | Convolutional Neural Network                      |
| COVID    | Coronavirus disease                               |
| CPU      | Central processing unit                           |
| CRFs     | Conditional random fields                         |
| DFT      | Discrete fourier transform                        |
| GFLOPs   | Giga floating point operations per second         |
| ELU      | Exponential linear unit                           |
| FC       | Fully connected                                   |
| FPGA     | Field-programmable gate array                     |
| GAN      | Generative adversarial network                    |
| GAP      | Global average pooling                            |
| GPU      | Graphics Processing Units                         |
| Grad-CAM | Gradient-weighted class activation mapping        |
| ILSVRC   | ImageNet large scale visual recognition challenge |
| KL       | Kullback-leibler                                  |
| LSTMs    | Long short-term memory networks                   |
| MACs     | Multiply–accumulate operations                    |
| MLPConv  | Multilayer perceptron convolution                 |
| MOP      | Multi-scale orderless pooling                     |
| NLP      | Natural language processing                       |
| PReLU    | Parametric rectified linear unit                  |
| PCA      | Principal component analysis                      |
| ReLU     | Rectified linear unit                             |
| RNN      | Recurrent neural network                          |
| RReLU    | Randomized rectified linear unit                  |
| SGD      | Stochastic gradient descent                       |
| SHAP     | Shapley additive explanations                     |
| SPP      | Spatial pyramid pooling                           |
| SSD      | Single shot MultiBox detector                     |
| t-SNE    | T-distributed stochastic neighbor embedding       |
| UMAP     | Uniform manifold approximation and projection     |
| VAEs     | Variational autoencoders                          |
| ViTs     | Vision transformers                               |
| VLAD     | Vector of locally aggregated descriptors          |
| YOLO     | You only look once                                |

## 1 Introduction

CNNs, a deep learning architecture, have transformed computer vision and other fields by enabling automatic feature extraction and hierarchical pattern identification. CNNs first emerged from study in neurology in 1959, when studies on the mammalian visual cortex showed that neurons react preferentially to visual patterns [1]. Fukushima developed the multi-layered Neocognitron in 1980, a multi-layered model able to learn hierarchical visual cues, hence laying the basis for contemporary CNN [2]. The first important use of CNNs was LeNet-5, developed by LeCun et al., who used backpropagation to train networks to recognize handwritten numbers in 1989 [3]. This marked a fundamental evolution of the notion of CNNs.

By the 2010s, CNNs were widely used in machine learning applications due to the availability of big, annotated datasets like ImageNet and improvements in computing power, especially Graphics Processing Units (GPUs). In the ImageNet Large Scale Visual Recognition Challenge (ILSVRC), Krizhevsky et al.'s AlexNet demonstrated that deep CNNs significantly outperformed traditional models in large-scale image classification tasks, achieving groundbreaking results [4].

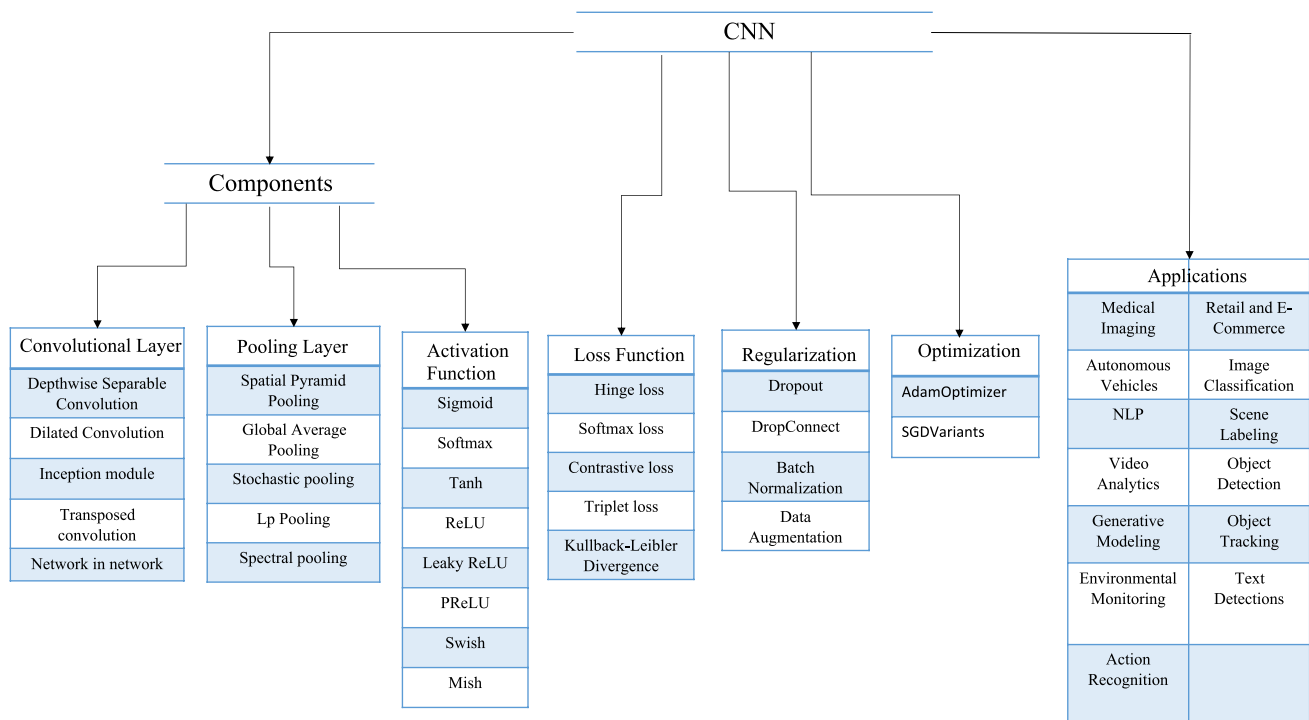
In order to overcome the difficulty of training intense networks, later models like ResNet introduced residual connections that prevent vanishing gradient problems and enable the stacking of hundreds of layers [5]. In the meantime, EfficientNet suggested a novel method for scaling CNNs that achieves high accuracy with fewer parameters by striking a balance between network width, depth, and resolution [6]. Architectural advancements, such as depthwise separable convolutions in MobileNet and Xception, have significantly reduced computational costs while maintaining performance [7].

Interpretability tools such as Grad-CAM are detailed in later sections to improve transparency in CNN decision-making, especially in sensitive areas like healthcare [8]. Hybrid designs, as detailed in a later section, such as CNN-RNN (Recurrent Neural Network) and CNN-Transformer models, are increasingly becoming more popular, particularly for applications that need complicated dependencies or sequential data [9]. CNN research is still developing as new architectures like Capsule Networks (CapsNet), which capture spatial relationships in data, lightweight models for edge devices, and efficient processing strategies are being investigated [10].

This review aims to provide a comprehensive synthesis of these developments by exploring the evolution of CNN architectures, analyzing enhancements in their structural components, and assessing their real-world applications. The review is intended to serve as a reference for researchers, engineers, and practitioners seeking to understand both foundational concepts and recent innovations in CNN design.

The following Fig. 1 shows the hierarchical overview of this paper.

The contributions of this study are as follows:



**Fig. 1** Hierarchical breakdown of CNN Architecture in this survey

1. Provides an overview of CNN fundamentals and classic models that have shaped the field, offering insights into their historical significance and foundational design principles.
2. Highlights Key advancements in CNN architecture, pooling strategies, and activation functions are analyzed to showcase their role in driving innovation.
3. Explores diverse applications of CNNs, illustrating their transformative impact on solving real-world challenges.
4. Explores potential avenues for future research, focusing on improving hardware efficiency, addressing model explainability and real-world deployment issues, and addressing new challenges in the deployment of CNNs.
5. A structured and comparative analysis is presented to simplify complex concepts and enhance understanding for readers.

The literature for this review was systematically selected from 1998–2024 using major databases (IEEE Xplore, SpringerLink, ScienceDirect, ACM Digital Library, arXiv). Priority was given to peer-reviewed works introducing impactful CNN architectures (e.g., LeNet, AlexNet, ResNet, EfficientNet), methodological advances (activation functions, pooling, training optimizations), and strong empirical results on benchmark datasets like ImageNet and CIFAR. Both foundational and recent state-of-the-art studies were included, while non-peer-reviewed or redundant works were excluded.

The structure of this paper is as follows: Sect. 2 provides a chronological literature review, summarizing major breakthroughs in CNN development. Section 3 elaborates on the fundamental architectural components, including convolutional layers, pooling techniques, activation functions, padding, loss functions, and optimization methods. Section 4 discusses significant improvements to CNN design and training, such as advanced convolution strategies, hybrid architectures, and novel regularization approaches. Section 5 explores diverse applications of CNNs across fields including medical imaging, autonomous systems, natural language processing, generative modeling, and environmental monitoring. Section 6 outlines the limitations of current CNN models and identifies key research directions for the future. Finally, Sect. 7 concludes the paper by summarizing major findings and insights.

## 2 Literature review

It is critical to study the important advancements and contributions in this discipline in order to comprehend the evolution of CNNs and their numerous applications. Table 1 lists the main developments in CNN research together with their publishing years, important results, special features, constraints, and citations. From early image classification to contemporary use cases including image synthesis, real-time video analysis, and hybrid architecture, Table 1 compiles the development of CNNs, including its applications.

## 3 Fundamental concepts of CNNs

### 3.1 CNN architecture

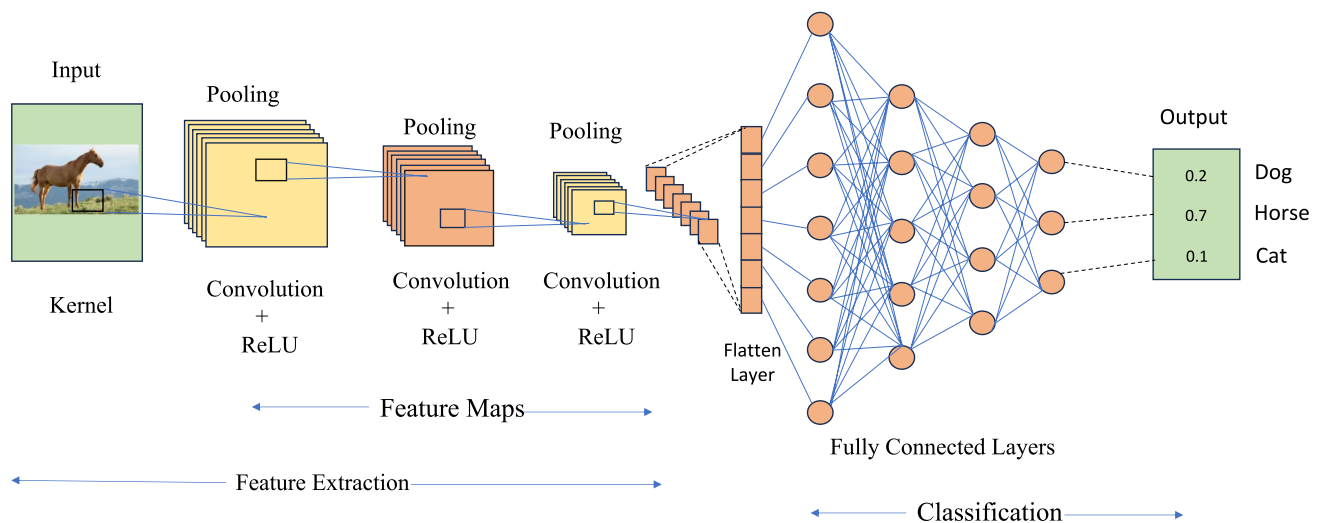
Figure 2 shows the basic architecture of CNN, each part of which is discussed in Sect. 3.2.

**Table 1** Overview of research progress in CNNs

| Paper | Pub-<br>lish-<br>ing<br>year | Summary                                                                                                        | Key findings                                                                                                        | Significant trait                                                                                  | Limitations                                                                                                                 |
|-------|------------------------------|----------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| [11]  | 1998                         | Introduced CNNs for image classification tasks; revolutionized computer vision applications                    | Demonstrated CNNs' ability to learn hierarchical features for image recognition                                     | Established convolutional layers, pooling, and activation functions as core components             | Struggled with vanishing gradients in deeper networks; required large, labeled datasets                                     |
| [12]  | 2013                         | Semi-supervised CNNs for label-scarce scenarios                                                                | Reduced dependence on labeled data; leveraged unlabeled sets                                                        | Proposed pseudo-labeling and contrastive learning for improved generalization                      | Sensitive to noisy labels; class imbalance issues                                                                           |
| [13]  | 2014                         | Explored transfer learning with CNNs for domain adaptation                                                     | Transfer learning reduces training time and data dependency; utilizes pre-trained models                            | Highlighted domain-specific fine-tuning for diverse applications like medical imaging and robotics | Limited flexibility with datasets that have completely unseen features                                                      |
| [5]   | 2016                         | ResNet was proposed as a solution to the deep CNN architecture's vanishing gradient issue                      | Residual connections enable extremely deep networks without performance degradation                                 | Introduced skip connections to improve gradient flow and facilitate network training               | Requires significant computational resources for training deeper networks                                                   |
| [14]  | 2017                         | CNNs based on Generative Adversarial Networks (GANs) were introduced for picture segmentation and synthesis    | Adversarial training enhances image realism and segmentation performance                                            | Combined generative and discriminative learning for enhanced output realism                        | Suffer from instability, require precise hyperparameter tuning                                                              |
| [15]  | 2017                         | Explored attention mechanisms in CNN architectures for enhanced feature focus                                  | Attention modules improve focus on significant regions in input data, enhancing performance                         | Self-attention processes that are integrated to capture long-range dependencies in data            | Increased computational overhead, complexity in model design                                                                |
| [16]  | 2017                         | Introduced DenseNet to improve gradient flow and feature reuse in deep CNNs                                    | Dense connections reduce parameter redundancy and enhance feature propagation                                       | Proposed densely connected layers to ensure maximum information flow across layers                 | Increased memory usage due to concatenated feature maps                                                                     |
| [10]  | 2017                         | Explored CapsNets for hierarchical feature detection                                                           | CapsNets capture spatial hierarchies and achieve equivariance in image analysis                                     | Introduced dynamic routing between capsules for better part-to-whole relationships                 | Computationally expensive compared to CNNs                                                                                  |
| [17]  | 2018                         | Utilized Darknet-53, a 53-layer CNN, for efficient feature extraction and multi-scale object detection         | Improved feature extraction and multi-scale detection; balanced speed and accuracy                                  | Incorporated residual connections and multi-scale detection using CNN feature maps                 | Difficulty detecting small objects in cluttered scenes; high computational demands; lacks spatiotemporal feature extraction |
| [18]  | 2018                         | Investigated adversarial robustness of CNNs in safety-critical applications                                    | Identified vulnerabilities of CNNs to adversarial attacks and proposed defense mechanisms                           | Analyzed adversarial training and input preprocessing to mitigate attacks                          | Limited generalizability of proposed defenses across different datasets                                                     |
| [19]  | 2019                         | EfficientDet builds upon EfficientNet and proposes a scalable, efficient object detection framework using CNNs | EfficientDet improves speed and accuracy of object detection using a CNN-based backbone                             | Combined EfficientNet with a novel compound scaling approach for object detection                  | High computational cost for large models, particularly in real-time applications                                            |
| [20]  | 2020                         | Developed Vision Transformers (ViTs) as an alternative to CNNs for visual tasks                                | ViTs outperform CNNs in large-scale image recognition tasks with sufficient data                                    | Eliminated the need for convolutional layers using transformer-based architectures                 | Requires extensive training data, high computational resource                                                               |
| [21]  | 2021                         | Introduced the Swin Transformer, a hierarchical model with shifted window attention                            | Outperforms CNNs and ViTs on several vision benchmarks, including image classification, detection, and segmentation | Hierarchical design with shifted windows for efficiency                                            | Requires extensive fine-tuning; significant computational resources                                                         |
| [22]  | 2023                         | Investigated complex-valued CNNs for brain tumor classification                                                | Complex-valued CNNs outperform traditional CNNs in medical imaging tasks                                            | Leverages complex features to capture intricate relationships in data                              | Demands higher computational resources due to an increased number of trainable parameters                                   |
| [23]  | 2023                         | Proposed a CNN architecture optimized for malware detection                                                    | Achieved 99.183% accuracy in malware detection                                                                      | Highlights the impact of model depth and optimization in cybersecurity                             | Performance may vary across different malware datasets, requires real-world validation                                      |

**Table 1** (continued)

| Paper | Pub-lish-ing year | Summary                                                                                          | Key findings                                                          | Significant trait                                                       | Limitations                                                                                     |
|-------|-------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|-------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| [24]  | 2023              | Combined 2D and 3D CNNs in a fusion model for predicting Coronavirus Disease (COVID)-19 severity | Fusion models outperform baseline approaches in medical imaging tasks | Combines 2D and 3D models for enhanced performance in CT scan analysis  | High computational resources required; hyper-parameter tuning needed                            |
| [25]  | 2024              | Comprehensive review of hybrid architectures combining CNNs and ViTs for computer vision tasks   | Highlighted potential of hybrid models; achieved superior performance | Surveyed hybrid designs and applications; insights into CNN–ViT synergy | Hybrid models add complexity and computational overhead; require careful integration strategies |

**Fig. 2** Basic CNN architecture showing feature extraction through convolution and pooling, followed by fully connected layers for classification [3]

## 3.2 Core components of CNN

### 3.2.1 Convolutional layer

In CNNs, the convolutional layer serves as the primary component for extracting hierarchical features from input data, particularly images. The foundation for deeper layers' higher-level abstractions is provided by convolutional layers, which, in contrast to completely linked layers, use filters (kernels) to identify patterns such as edges, textures, and forms. In tasks like image classification and object detection, these layers leverage spatial proximity by preserving relationships between adjacent pixels.

A mathematical description of the function carried out in a convolutional layer is as follows:

$$Z_{i,j}^{(k)} = \sum_{m=1} \sum_{n=1} X_{(i+m-1)(j+n-1)} \cdot W_{m,n}^{(k)} + b^{(k)} \quad (1)$$

where  $X$ : Input matrix (such as a feature map or picture),  $W$ : Filter matrix of size  $M \times N$  (kernel),  $b$ : The bias term,  $Z$ : The feature map output that corresponds to filter  $k$  and  $i, j$ : The generated feature map's spatial location indicators.

Here, element-wise multiplication is carried out at each point as the filter  $W$  moves across the input  $X$ . The products are summed, and the bias  $b$  is added to the computed value, enhancing the model's expressiveness [7].

To enable the network to capture intricate patterns,  $Z$  is then subjected to a non-linear activation function, such as the Rectified Linear Unit (ReLU).

Consider a  $4 \times 4$  input  $X$ , a  $2 \times 2$  filter  $W$ , stride of 1, and no padding:

$$\text{Input } X : \begin{bmatrix} 1 & 2 & 0 & 1 \\ 3 & 1 & 1 & 0 \\ 1 & 2 & 2 & 3 \\ 0 & 1 & 3 & 2 \end{bmatrix} \quad \text{Filter } W : \begin{bmatrix} 1 & 0 \\ -1 & 0 \end{bmatrix}$$

For the top-left region:

$$Z_{1,1} = (1 \cdot 1) + (2 \cdot 0) + (3 \cdot -1) + (1 \cdot 0) = 1 + 0 - 3 + 0 = -2$$

Repeating this for all regions produces the output feature map, as shown in Fig. 3.

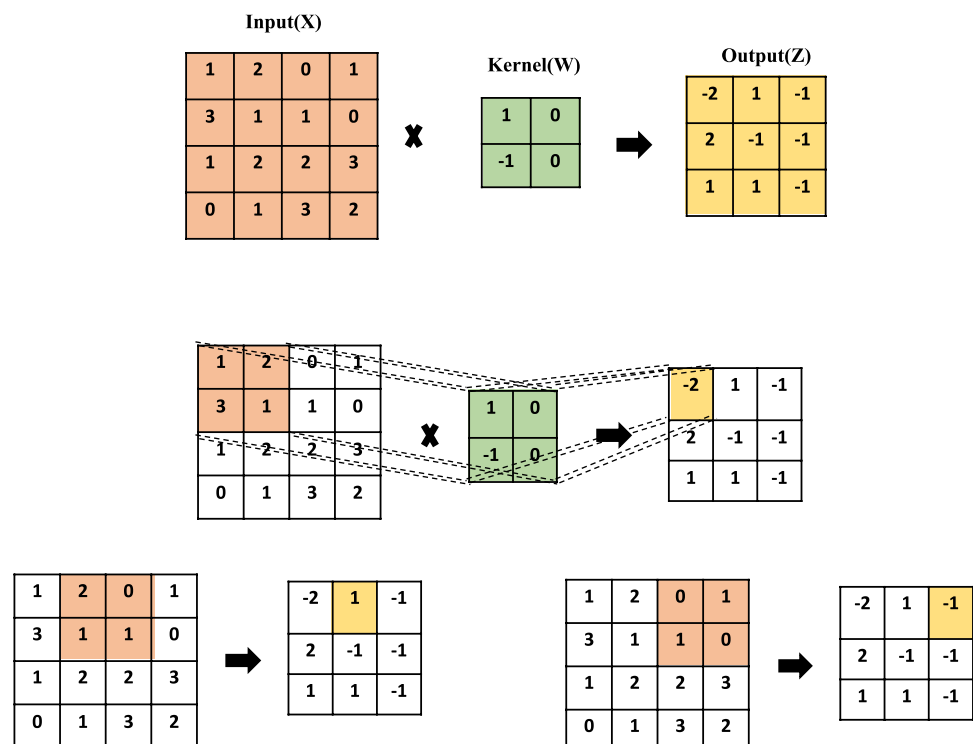
Convolutional layers play a part in creating scalable systems in addition to feature extraction. For instance, CNNs like AlexNet demonstrated the effectiveness of convolutional layers on large-scale datasets by incorporating innovations such as ReLU activation and dropout. Later architectures such as ResNet, which introduced residual connections to address vanishing gradients, and VGGNet, known for its deep architecture with uniform  $3 \times 3$  convolutional layers, further emphasized the importance of convolutional layers [26].

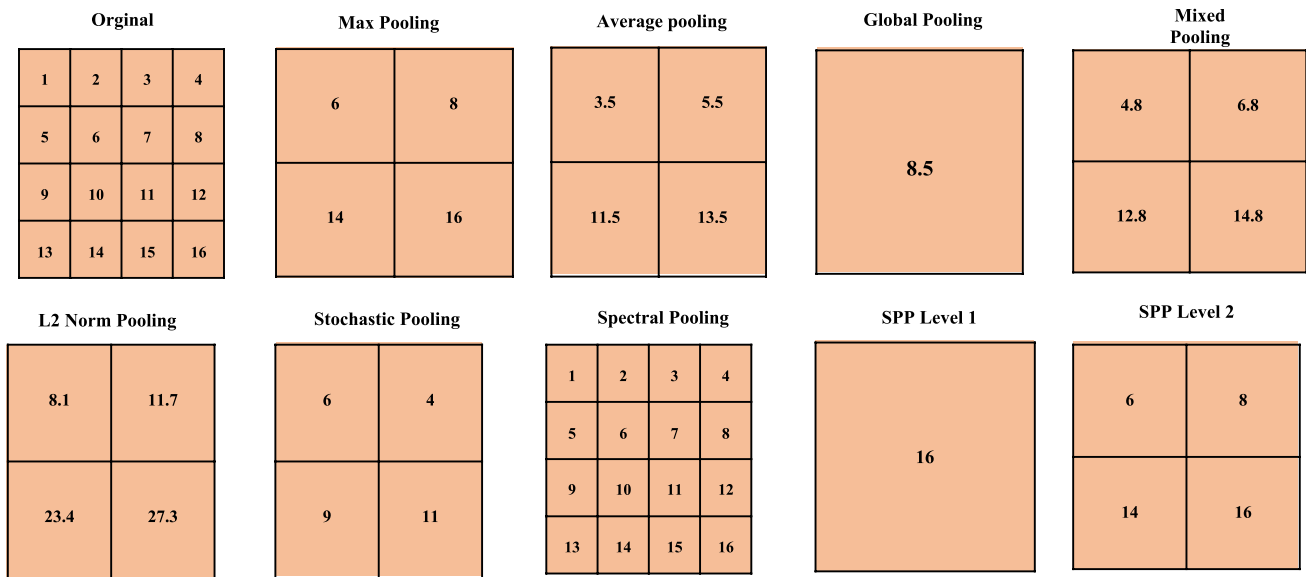
### 3.2.2 Pooling layer

In CNNs, pooling layer is an essential component. In order to downsample the data while keeping important information, it shrinks the feature maps' spatial dimensions (height and width). By summarizing the presence of features in local regions, pooling reduces computational cost, minimizes memory usage, and helps prevent overfitting.

Building translation-invariant models requires pooling since it concentrates on a feature's presence rather than its precise placement within an area. Modern CNN architectures like AlexNet [4] and ResNet [5] rely on pooling,

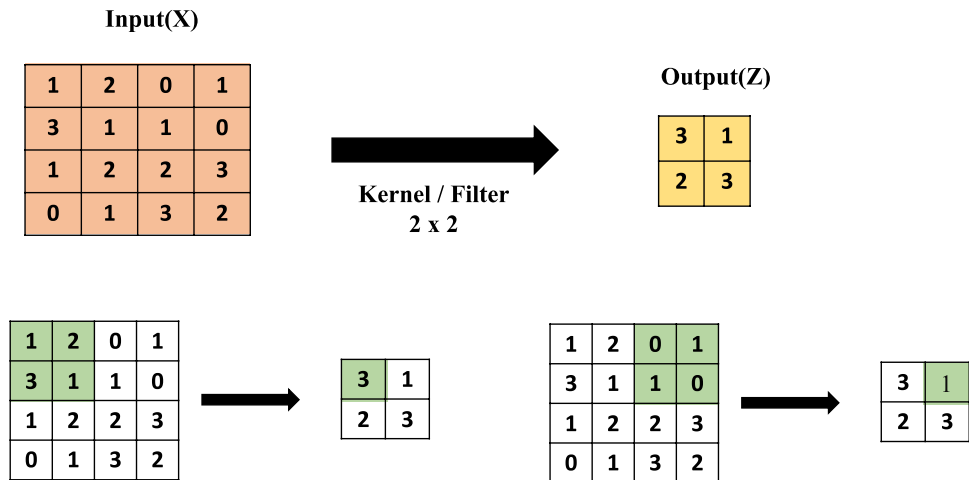
**Fig. 3** A CNN's convolution operation creating the output feature map by convolving the input matrix with the kernel [11]





**Fig. 4** Max Pooling, Average Pooling, Global Pooling, Mixed Pooling, L2 Norm Pooling, Stochastic Pooling, Spectral Pooling, Spatial Pyramid Pooling (SPP) Level1, SP level 2 Pooling on a 4×4 Array

**Fig. 5** Max Pooling Operation in a CNN Using a 2×2 Kernel



a concept utilized in early CNNs such as LeNet-5 by Yann LeCun et al. [11], though in a different form as sub-sampling layers. Some pooling methods used in CNNs (Visualization shown in Fig. 4):

**3.2.2.1 Max pooling** One of the most popular pooling strategies is max pooling. By choosing the highest value in each pooling window, it shrinks the size of a feature map. Max pooling was first introduced by Yamaguchi et al. for speaker-independent word recognition [27]. While it was initially developed for speech recognition tasks, max pooling gained widespread adoption in computer vision after being prominently featured in AlexNet [197], which demonstrated its effectiveness in improving CNN performance. Zeiler and Fergus [28] further analyzed the role of max pooling in enhancing feature visualization and understanding CNN behavior. For an input feature map  $X$  of size  $h \times w$  and a pooling window of size  $p \times p$  the max-pooled output  $P_{i,j}$  is computed as:

$$P_{ij} = \max_{m=0}^{p-1} \max_{n=0}^{p-1} X_{(i+m)(j+n)} \quad (2)$$

In this case,  $P_{ij}$ . Within the pooling window, the input values are  $X_{(i+m)(j+n)}$  and the max-pooled value at position  $(i,j)$ . The max pooling technique is depicted in Fig. 5. In applications like object and edge recognition, where the model must concentrate on the existence of particular features, max pooling is frequently utilized.

**3.2.2.2 Average pooling** Average pooling computes the average of all pixel values within the pooling window. Compared to max pooling, it retains more information about the distribution of characteristics in the region. The average pooling method in CNNs, first explored by LeCun et al. in 1989 [3] and implemented in the LeNet-5 model as subsampling, down-samples input feature maps by averaging values within a pooling window. This method enhances computational efficiency and improves generalization.

For a feature map  $X$  and a pooling window of size  $p \times p$ , the average-pooled output  $P_{ij}$  is:

$$P_{ij} = \frac{1}{P \times P} \sum_{m=0}^{p-1} \sum_{n=0}^{p-1} X_{(i+m)(j+n)} \quad (3)$$

If  $m$  and  $n$  iterate across the pooling window of size  $p \times p$ , then  $X_{(i+m)(j+n)}$  is the value in the input feature map at position  $(i+m, j+n)$ , and  $P_{ij}$  is the pooled output value at position  $(i,j)$  in the output feature map. The size of the pooling window,  $p \times p$ , is typically  $2 \times 2$ ,  $3 \times 3$ , etc., and the two sums continuously cycle across it. This method is particularly useful when preserving all feature information is essential. Figure 6 shows a visualization of the average pooling operation.

### 3.2.2.3 Global pooling

Global pooling techniques, such as global average pooling (GAP) and global max pooling, are widely used in CNNs to improve generalization, reduce feature map dimensionality, and replace fully connected layers in deep architectures.

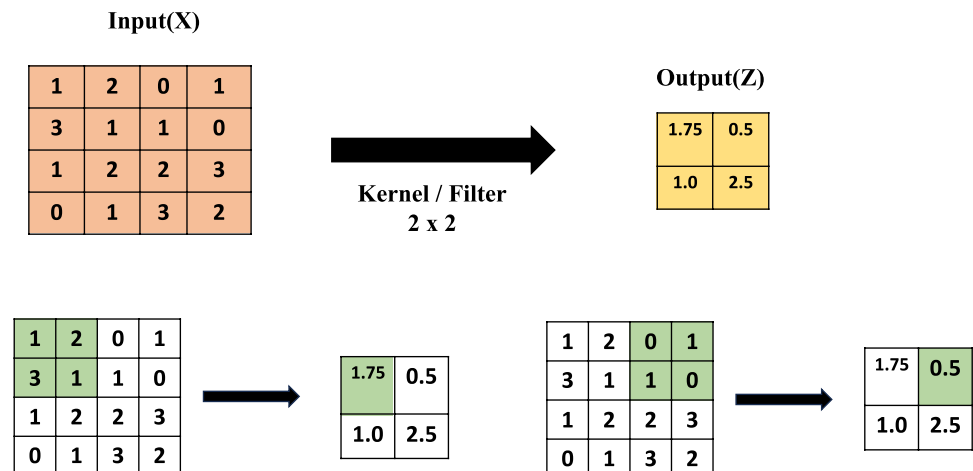
#### 1. Global max pooling

Global max pooling [29] extracts the most prominent feature by selecting the maximum value across the entire feature map. This method provides a compact representation and is particularly useful for emphasizing dominant features in the input data.

The equation for Global Max Pooling:

$$P = \max_{i,j} X_{ij} \quad (4)$$

**Fig. 6** Average Pooling Operation in a CNN Using a  $2 \times 2$  Kernel



When  $X_{ij}$  stands for the input matrix or feature map's elements,  $\max(X_{ij})$  means that we can use global max pooling to extract the maximum value from each element in  $X$  across all rows and columns (indexed by  $i$  and  $j$ ).

## 2. GAP

GAP [30] is a method to replace fully connected layers in neural networks, improving the model's generalization and reducing overfitting by compressing the spatial dimensions of the feature maps into a single scalar value per feature map. Equation:

$$P = \frac{1}{h \times w} \sum_{i=1}^h \sum_{j=1}^w X_{ij} \quad (5)$$

where  $P$  is the output scalar value,  $h$  and  $w$  are the height and width of the feature map, respectively,  $X_{ij}$  represents the element of the feature map at position  $(i, j)$  where,  $i$  and  $j$  correspond to the row and column indices of the feature map.

By lowering spatial dimensions to  $1 \times 1$ , global pooling significantly lowers model parameters. For classification tasks, it is very helpful when feature maps are combined into a single vector.

**3.2.2.4 Mixed pooling** Mixed pooling is a hybrid approach that combines max pooling and average pooling. Studies by [84, 196] have shown that mixed pooling enhances feature-extraction robustness in CNNs, leading to improved image classification performance [31].

In Mixed Pooling, for each pooling window, a random choice is made between max pooling and average pooling:

$$P_{ij} = \begin{cases} \max(X_{(i+m)(j+n)}) & \text{with probability } p \\ \frac{1}{P \times P} \sum X_{(i+m)(j+n)} & \text{with probability } 1 - p \end{cases} \quad (6)$$

here  $X_{(i+m)(j+n)}$  Represents elements of the feature map from a sliding window over the input feature map centered at position  $(i, j)$ ,  $m$ , and  $n$  index neighboring elements within the pooling window.  $\max(X_{(i+m)(j+n)})$  selects the maximum value within the pooling window with probability  $p$ . If  $p=0.5$ , max pooling and average pooling have equal chances of being applied. Mixed pooling provides robustness by combining the benefits of both methods and reduces the likelihood of overfitting compared to using a single pooling method.

**3.2.2.5 L2 norm pooling** L2 norm pooling aggregates squared activations over a pooling region, enhancing robustness to feature variations in visual recognition tasks [4]. For a pooling window of size  $p \times p$ , the L2-pooled value  $P_{ij}$  is:

$$P_{ij} = \sqrt{\sum_{m=0}^{p-1} \sum_{n=0}^{p-1} X_{(i+m)(j+n)}^2} \quad (7)$$

In this case, The value of a pixel in the feature map inside the pooling window is represented by the formula  $X_{(i+m)(j+n)}$ . Starting at  $(i, j)$ , the indices  $(i+m)$  and  $(j+n)$  define the location in the feature map with respect to the upper-left corner of the pooling window. The sums compute the sum of the squares of all the values in the  $p \times p$  pooling window, which is a square region of size  $p \times p$  that loops over the height and width of the window and is determined by  $m$  and  $n$ . The result of the summation is subjected to the square root. The pooling window's L2 norm is the result of  $P_{ij}$ . After that, the output feature map's matching position is given this value.

**3.2.2.6 Stochastic pooling** In order to improve generalization in CNNs, Zeiler, and Fergus [28, 32] devised stochastic pooling, which selects activations at random according to their probabilities and yields reliable results against overfitting.

Unlike deterministic techniques like max pooling or average pooling, this stochastic selection adds unpredictability to the pooling process. Stochastic Pooling improves generalization by introducing randomness and avoids over-reliance on the strongest feature (as in max pooling).

**3.2.2.7 Spectral pooling** In contrast to conventional spatial pooling, spectral pooling uses the Fourier domain to decrease spatial resolution while maintaining more information, resulting in better performance on picture classification tasks [33]. It is a frequency-domain pooling method introduced by Rippel et al. [34]. By truncating the feature map's frequency spectrum in the Fourier domain, this pooling technique lowers spatial dimensions while preserving more global and structural information than spatial pooling techniques.

**3.2.2.8 Spatial pyramid pooling (SPP)** SPP enhances object detection and image classification by enabling multi-scale feature learning. This was demonstrated in SPP-net on the Pascal VOC 2007 and ImageNet datasets [35]. Spatial Pyramid Pooling divides the feature map into bins of varying sizes and performs pooling (e.g., max or average) in each bin. It gathers multi-scale spatial information and can handle variable-sized inputs, which makes it ideal for object detection [36].

**3.2.2.9 Multi-scale orderless pooling (MOP)** By combining data from several scales without requiring spatial ordering, MOP has been demonstrated to improve feature extraction and performance on tasks like picture classification and retrieval. It was proposed by Gong et al. in 2014 [37]. MOP uses orderless pooling techniques such as bag-of-words or VLAD (Vector of Locally Aggregated Descriptors) to aggregate features that are extracted from various sizes of a feature map. The MOP captures scale-invariant features and retains robust feature representations without spatial constraints.

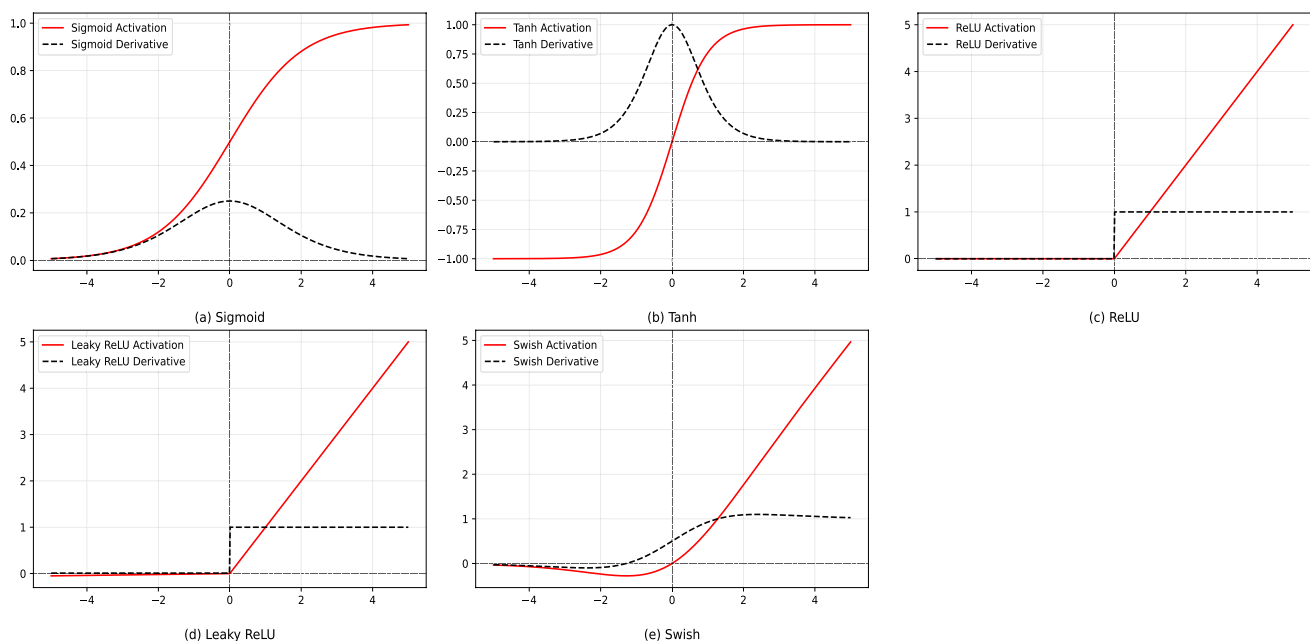
For the mathematical formulations of the advanced pooling techniques in Sects. 3.2.2.6 to 3.2.2.9, refer to Appendix 1.

### 3.2.3 Activation functions

In order to learn intricate relationships and patterns in data, activation functions—which introduce non-linearity—are crucial parts of neural networks. [38]. They have undergone substantial development to tackle issues including computational efficiency, model robustness, and vanishing gradients [39]. The CNN activation functions are introduced in this section.

Figure 7 shows a graphical representation of Activation Functions—sigmoid, Tanh, ReLU, Leaky ReLU, Swish.

**3.2.3.1 Sigmoid** One of the first activation functions, the sigmoid function, was popularized by Rumelhart, Hinton, and Williams in their groundbreaking study on backpropagation [40]. The function is helpful when the result



**Fig. 7** Graphical representation of Sigmoid, Tanh, ReLU, Leaky ReLU, Swish Activation functions and their Derivative

needs to be understood as a probability because it compresses its input into a range between 0 and 1. This is how the sigmoid function is defined:

$$F(x) = \frac{1}{1 + e^{-x}} \quad (8)$$

where  $x$  is the input to the function (a weighted sum of inputs to a neuron). Sigmoid maps inputs to a range (0, 1), with a smooth gradient given by  $f'(x) = f(x) \cdot (1 - f(x))$ . The vanishing gradient problem, which makes gradients insignificant for large positive or negative inputs, makes deep network training difficult.

**3.2.3.2 Tanh** Tanh [38], a scaled and shifted version of the sigmoid function, was developed to address Sigmoid's limitations, particularly its non-zero-centered output [44]. Tanh speeds up convergence during training by generating zero-centered outputs. The formula for the tanh activation function is:

$$F(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \quad (9)$$

Its gradient is:  $f'(x) = 1 - f(x)^2$ . It has a similar vanishing gradient problem to sigmoid in that it decreases for extreme inputs. When a balanced data distribution is required, Tanh outperforms Sigmoid [41].

**3.2.3.3 ReLU** ReLU, proposed by Nair and Hinton, became a cornerstone in deep learning due to its simplicity and computational efficiency [42]. It helps introduce non-linearity into the model, which is essential for learning complex patterns. The mathematical definition of the ReLU function is:

$$F(x) = \max(0, x) \quad (10)$$

This function produces  $x$  if  $x > 0$ , otherwise 0. Because fewer neurons are engaged, this sparsity adds computing efficiency. For  $x > 0$ , the gradient remains constant, effectively mitigating the vanishing gradient problem. The 'dying ReLU' problem occurs when gradients become zero for  $x \leq 0$ , causing neurons to become permanently inactive. ReLU enables faster training and better generalization compared to sigmoid and tanh [43].

**3.2.3.4 Randomized rectified linear unit (RRReLU)** Xu et al. proposed RRReLU, which uses random negative slopes during training to improve generalization and robustness [44]. It is a variation of the activation function of ReLU. The primary distinction between RRReLU and ReLU is that RRReLU adds randomness for negative values in place of a set slope. Mathematically:

$$F(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha x & \text{if } x \leq 0 \end{cases} \quad (11)$$

here  $\alpha$  is a random value chosen from a uniform distribution within a given range (usually between a lower and upper bound). RRReLU outputs the same value as ReLU when the inputs are positive. A random slope, taken from a uniform distribution within a given range, is introduced for negative inputs by RRReLU. This lessens the possibility of "dying ReLU" issues, in which neurons may stop learning and go dormant [44].

**3.2.3.5 Leaky ReLU** A modified form of the ReLU function, known as the Leaky ReLU activation function, is used to solve the problem of "dying neurons" in neural networks, which occur when neurons cease learning and become inactive. It was introduced by Maas et al. [43]. The Leaky ReLU function is defined as:

$$F(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha x & \text{if } x \leq 0 \end{cases} \quad (12)$$

But here  $\alpha$  is a constant (e.g., 0.01) that determines the slope of the function for negative values of  $x$ . To keep neurons alive, outputs  $x$  for positive inputs and  $\alpha x$  for negative inputs. When the input is negative, Leaky ReLU permits a tiny, non-zero gradient, which is essential for preserving efficient gradient flow in deeper networks and preventing problems with gradient vanishing during training [35].

**3.2.3.6 Parametric rectified linear unit (PReLU)** In order to overcome the constraints of regular ReLU and improve model flexibility, PReLU, a version of ReLU, was proposed to make the activation function learnable by adding a parameter that modifies the slope for negative inputs [45]. The PReLU function is defined as:

$$F(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha x & \text{if } x \leq 0 \end{cases} \quad (13)$$

where  $\alpha$  is a learnable parameter that is optimized during training. Because of its learnable nature, the network may adaptively optimize the negative part's slope ( $\alpha$ ) for every layer. For positive inputs, PReLU behaves like ReLU by outputting the input value. For negative inputs, the parameter  $\alpha$  determines the slope, which is learned dynamically during backpropagation. In contrast to ReLU, PReLU adds learnability, which improves model accuracy in jobs with widely differing data distributions. Avoiding manual adjustment of the negative slope parameter allows it to outperform Leaky ReLU [46].

**3.2.3.7 Exponential linear unit (ELU)** ELU was created to improve learning dynamics while resolving the dying ReLU issue and vanishing gradient problem [47]. It extends ReLU by introducing exponential behavior for negative inputs and offers a smooth, differentiable function.

ELU beats Leaky ReLU and PReLU in situations where the exponential behavior of negative inputs helps with regularization and feature extraction, and it performs better than ReLU in jobs with noisy data or imbalanced features because of its smooth gradient flow for negative inputs [48].

**3.2.3.8 Maxout** An alternate nonlinear function called Maxout determines the maximum response across several channels at every geographical location [49]. Maxout calculates the maximum value from a number of linear functions rather than applying only one (as in ReLU). Because of this, it may depict more intricate patterns than basic activation functions. The network can determine the best activation for every neuron by parameterizing each linear function with a set of weights and biases.

**3.2.3.9 Probout** A probabilistic version of ReLU called Probout was created to add stochastic behavior to the activation function [50]. Probout adds stochastic behavior to deep learning models, which can improve their robustness and generalization when compared to more deterministic activations (like ReLU and Maxout). Nevertheless, this results in a higher computational complexity [51].

## Swish

A smooth, non-monotonic activation function called Swish was developed by Google researchers and has been demonstrated to perform better than ReLU in a variety of deep-learning tasks [52]. It integrates the benefits of ReLU and sigmoid to produce a continuous, differentiable function that works effectively in very deep neural networks. Swish's non-monotonicity allows it to capture more complex patterns than ReLU and other activation functions. [53].

For the mathematical formulations of the advanced activation functions in Sects. 3.2.3.7 to 3.2.3.10, refer to Appendix 1.

### 3.2.4 Padding

Padding in CNNs is used to control the output feature map size and preserve edge information during convolution operations. It involves adding extra pixels, typically zeros, around the input matrix [54]. It is essential in creating deeper networks and maintaining spatial dimensions. In the following section, some padding methods are discussed in brief.

**3.2.4.1 Valid padding (No padding)** In CNNs, valid padding, also called no padding, makes sure that no extra pixels are added to the input image before applying convolution operations, which reduces the output's spatial dimensions relative to the input [28]. Valid padding is commonly used when the goal is to progressively reduce feature map dimensions, making precise size retention unnecessary [45]. The formula for output size:

$$\text{Output Size} = \frac{\text{Input Size} - \text{Filter Size} + 1}{\text{Stride}} \quad (14)$$

It concentrates on valid regions to facilitate effective hierarchical feature abstraction and ever-smaller feature maps for localized pattern detection. As a result, it supports deeper and more effective network designs and is perfect for object identification and classification tasks [55].

**3.2.4.2 Same padding** Same padding ensures that the height and width of the output feature map remain identical to those of the input feature map in CNNs [56]. In order to accomplish this, the input image is padded with zeros (or occasionally other values) so that the convolution operation can cover all area of the input, including the edges, without lowering the output size [57]. For an input image of size  $H \times W$  and a convolution kernel of size  $F \times F$ , the same padding ensures that the output feature map size remains the same as  $H \times W$  after applying the convolution. This requires adding padding to the borders of the input image. The padding is typically calculated as:

$$P = \frac{F - 1}{2} \quad (15)$$

where  $P$  is the padding added to each side (top, bottom, left, right) of the input image. For odd-sized kernels, the padding will be symmetric, while for even-sized kernels, the padding might not be perfectly symmetric, but it still ensures the output size matches the input size [58].

**3.2.4.3 Full padding** In order to guarantee that the convolutional filter can cover every pixel, including the edges, full padding adds enough padding around the input matrix [59]. As a result, the output feature map is greater than the input. The output size  $O$  for a convolution with full padding can be calculated as:

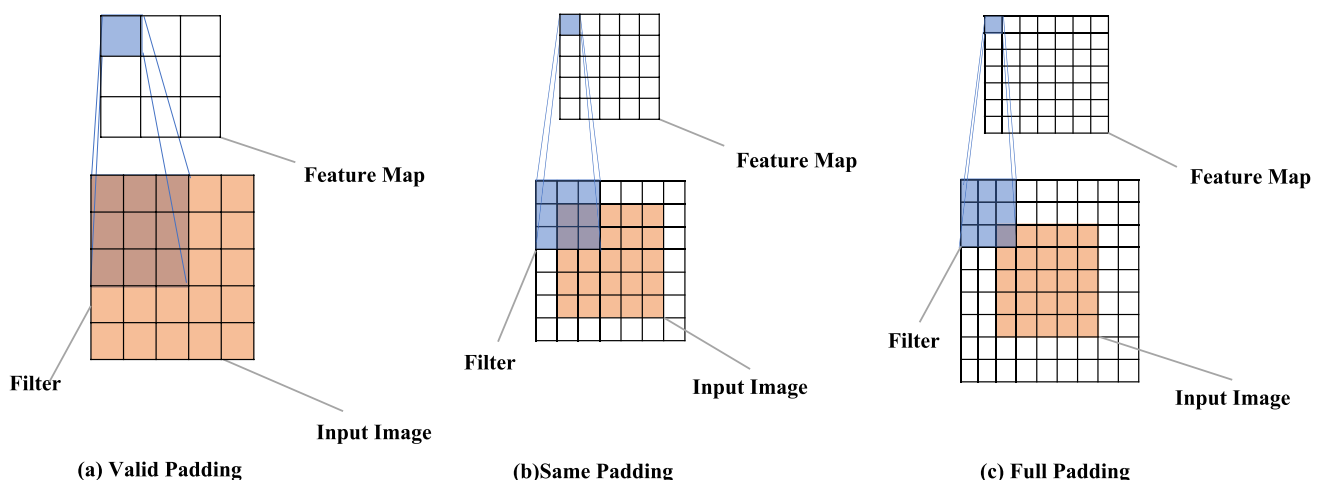
$$O = \frac{I + 2P - F}{S} + 1 \quad (16)$$

where  $I$  is the input size,  $P$  is the padding,  $F$  is the filter size, and  $S$  is the stride. For full padding, the padding  $P$  is calculated as:

$$P = F - 1 \quad (17)$$

This guarantees that the output feature map will be larger than the input and that the filter can cover the entire input, including the border [58]. Figure 8 shows a visualization of Padding—(a) Valid Padding, (b) Same Padding, (c) Full Padding.

The following Table 2 presents a comparison of different padding techniques, namely Valid Padding, Same Padding, and Full Padding.

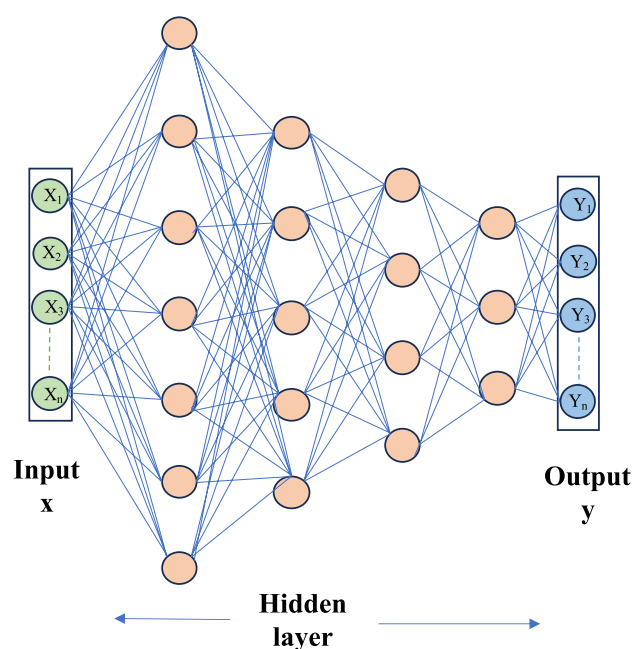


**Fig. 8** Different kinds of padding (valid, same, Full) techniques in CNN

**Table 2** Comparison of different padding types

| Padding type  | Description                                                                          | Mathematical formula for P | Impact on feature map size                                           | Filter coverage                                       | Computational cost | Use cases                                                                                 | Citations |
|---------------|--------------------------------------------------------------------------------------|----------------------------|----------------------------------------------------------------------|-------------------------------------------------------|--------------------|-------------------------------------------------------------------------------------------|-----------|
| Valid padding | No extra pixels are added to the input                                               | $P=0$                      | Compared to the input feature map, the output feature map is smaller | Center regions only; borders ignored                  | Lowest             | Suitable for tasks where edge information is irrelevant (e.g., central object detection)  | [60]      |
| Same padding  | To guarantee that the output feature map size matches the input, padding is provided | $P = \frac{F-1}{2}$        | Output feature map size remains unchanged                            | Full input, including borders                         | Moderate           | Widely used in image recognition tasks where input–output size consistency is needed      | [61]      |
| Full padding  | The padding ensures that the filter can cover the entire input, including borders    | $P=F-1$                    | Output feature map is larger than the input                          | Full input extends coverage beyond the original input | Highest            | Used in tasks requiring extended feature coverage, like object detection and segmentation | [58]      |

**Fig. 9** FC Layer in a CNN linking input features to output predictions through dense connectivity



### 3.2.5 Fully connected (FC) layer

The FC layer (Shown in Fig. 9) serves as the last location where features gathered by the convolutional and pooling layers are aggregated. It creates global feature representations for tasks like regression or classification by linking each neuron in one layer to every other layer [62]. Unlike convolutional layers that learn spatial hierarchies, the FC layer focuses on integrating extracted features and mapping them to decision spaces. For an input vector  $x \in \mathbb{R}^n$ , weight matrix  $W \in \mathbb{R}^{m \times n}$ , bias vector  $b \in \mathbb{R}^m$  and output vector  $z \in \mathbb{R}^m$ , the computation of FC layer is expressed as:

$$Z = Wx + b \quad (18)$$

here  $W$  represents the trainable weights, and  $b$  introduces translation. This dense representation is essential for tasks like classification, where an activation function such as Softmax is applied to  $z$  to yield probabilistic outputs across classes.

FC layers serve as a link between task-specific outputs and feature extraction. To avoid overfitting, AlexNet used multiple FC layers with dropout and ReLU [63]. FC layers are still necessary for jobs requiring high-dimensional feature generation, as demonstrated by DenseNet and EfficientNet [16]. However, they are frequently replaced by global pooling in modern designs like ResNet and Inception to minimize parameters.

FC layers' dense connectivity enables learning complex non-linear transformations, critical for challenging tasks [49]. However, this comes at the expense of higher parameter counts and potential overfitting, mitigated by methods like dropout, Batch Normalization (BN), and pruning [64].

### 3.2.6 Loss functions

A crucial part of CNNs are loss functions, which measure the difference between expected and actual outputs and efficiently direct the optimization process. Choosing the right loss function is essential for achieving optimal CNN performance.

**3.2.6.1 Hinge loss** Many large-margin classifiers, including Support Vector Machines, incorporate hinge loss. [64]. It penalizes incorrect predictions that lie within a certain margin from the decision boundary, ensuring better separation between classes. At the last layer of CNNs, hinge loss can take the place of more conventional loss functions like softmax loss. It works effectively for jobs that need robust classification and distinct decision boundaries because it enforces large-margin restrictions, which encourage well-separated class boundaries and decrease overfitting [65]. The loss function is defined as:

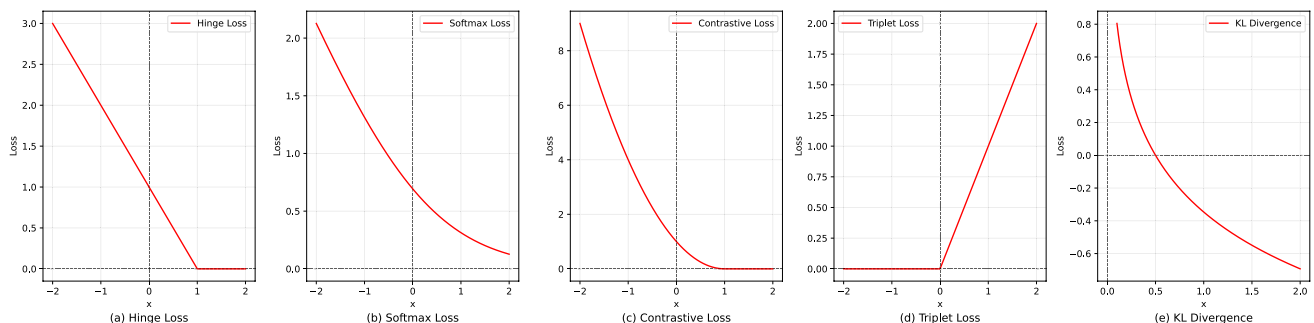
$$L = \frac{1}{N} \sum_{i=1}^N \max(0, 1 - \delta(y_i, j) z_j) \quad (19)$$

where  $z_j$  is the output score for class  $j$ , and  $\delta(y_i, j)$  equals 1 if  $j=y_i$  and -1 otherwise. Figure 10(a) shows the graphical representation of Hinge loss. Hinge loss has been widely used in SVM-based deep learning models to enhance classification accuracy.

**3.2.6.2 Softmax loss** Softmax loss works effectively for multi-class classification applications because it combines cross-entropy loss with the softmax activation function. The softmax function computes the predicted probability for class  $j$ , ensuring the output forms a valid probability distribution over all classes [61]. This makes softmax loss particularly effective for models like CNNs, where probabilistic predictions and accurate class probabilities are required for classification [66]. The predicted probability for a class  $j$  is calculated using the softmax function:

$$P_j = \frac{e^{z_j}}{\sum_{k=1}^C e^{z_k}} \quad (20)$$

and the loss is defined as,



**Fig. 10** Graphical representation of different types of Loss Functions

$$L = -\frac{1}{N} \sum_{i=1}^N \log(p_{y_i}) \quad (21)$$

where  $p_{y_i}$  is the predicted probability for the true class label  $y_i$ . Figure 10b shows the graphical representation of Softmax loss. Softmax loss is extensively used in tasks such as image classification and natural language processing.

**3.2.6.3 Contrastive loss** Contrastive loss is commonly used in similarity learning tasks, such as face verification, where the objective is to efficiently distinguish between similar and dissimilar pairings [67]. CNN-based Siamese networks, which construct embeddings for input pairs in order to learn a meaningful distance measure, benefit greatly from this loss function [68]. Contrastive loss encourages the network to generate closely aligned representations by reducing the Euclidean distance between embeddings for similar pairs. The loss is described as follows:

$$L = \frac{1}{N} \sum [y \cdot d^2 + (1 - y) \cdot \max(0, m - d)^2] \quad (22)$$

where  $d$  is the Euclidean distance between embeddings,  $y$  indicates whether the pair is similar ( $y=1$ ) or dissimilar ( $y=0$ ), and  $m$  is the margin. Figure 10(c) shows the graphical representation of Contrastive loss. Contrastive loss has shown excellent performance in Siamese networks for metric learning.

**3.2.6.4 Triplet loss** Another key metric learning loss function is triplet loss, which uses triplets—an anchor, a positive example, and a negative example—to improve embeddings [69]. This loss motivates the model to make sure the anchor is at least a margin  $m$  closer to the positive example than the negative example [70]. This is accomplished, subject to the margin constraint, by maximizing the distance between the anchor and the negative and decreasing the distance between the anchor and the positive. The loss is expressed as:

$$L = \frac{1}{N} \sum \max(0, d(a, p) - d(a, n) + m) \quad (23)$$

where  $d(a, p)$  and  $d(a, n)$  represent distances between the anchor-positive and anchor-negative pairs, respectively. Figure 10d shows the graphical representation of Triplet loss. Triplet loss is extensively used in applications such as face recognition and person re-identification, where learning discriminative embeddings is crucial.

**3.2.6.5 Kullback–Leibler divergence** Kullback–Leibler (KL) divergence is a fundamental measure for comparing two probability distributions:  $P(x)$  representing the true distribution, and  $Q(x)$  representing the estimated distribution [71]. KL divergence is very helpful in probabilistic models because, in contrast to typical loss functions, it measures how closely the projected distribution resembles the true distribution [72]. The loss is defined as:

$$L = \sum P(x) \cdot \log\left(\frac{P(x)}{Q(x)}\right) \quad (24)$$

or equivalently:

$$L = -H(P) - E_P[\log(Q(x))] \quad (25)$$

where  $H(P)$  is the entropy of  $P(x)$ . Figure 10(e) shows the graphical representation of Kullback–Leibler Divergence. KL divergence is often used in variational autoencoders (VAEs) and Bayesian neural networks to regularize learned distributions.

### 3.2.7 Optimizer

Optimizers are essential in CNNs as they adjust model parameters to minimize the loss function and improve learning efficiency [73]. They guide the learning process by computing gradients during backpropagation and changing weights for efficient convergence.

Types of Optimizers:

#### 3.2.7.1 Gradient descent variants

##### 1. Stochastic Gradient Descent (SGD)

A classical optimizer that updates parameters using a single data sample per iteration. While computationally efficient, it suffers from high variance in updates, leading to oscillations during convergence [74]. SGD updates parameters using a single data sample at each iteration:

$$\Theta_{t+1} = \Theta_t - \eta \nabla_{\Theta} J(\Theta_t) \quad (26)$$

where  $\Theta_t$  are the parameters at step  $t$ ,  $\eta$  is the learning rate, and  $\nabla_{\Theta} J(\Theta_t)$  is the gradient of the loss function  $J$  with respect to  $\Theta$ .

##### 2. Mini-Batch Gradient Descent

Mini-batch gradient descent balances the advantages of batch and stochastic methods by updating parameters using a small subset of the dataset, leading to more stable updates [75]. This optimizer updates parameters using a small batch of  $m$  samples:

$$\Theta_{t+1} = \Theta_t - \eta \frac{1}{m} \sum_{i=1}^m \nabla_{\Theta} J_i(\Theta_t) \quad (27)$$

##### 3. Momentum SGD

This optimizer accelerates convergence and aids in overcoming local minima by adding a momentum factor to smooth updates [76]. The mathematical expression of this optimizer:

$$V_t = \gamma v_{t-1} + \eta \nabla_{\Theta} J(\Theta_t), \quad \Theta_{t+1} = \Theta_t - v_t \quad (28)$$

where  $v_t$  is the velocity and  $\gamma$  is the momentum coefficient.

#### 3.2.7.2 Adaptive Optimizers

##### 1. Adagrad

Adagrad dynamically adjusts the learning rate for each parameter based on historical gradients, making it effective for sparse data. However, it suffers from a continuously decreasing learning rate, which can hinder long-term training [77]. Adagrad adjusts the learning rate based on past gradients:

$$\Theta_{t+1} = \Theta_t - \frac{\eta}{G_t + \epsilon} \nabla_{\Theta} J(\Theta_t) \quad (29)$$

where  $G_t = \sum_{i=1}^t g_i^2$  is the accumulation of squared gradients, and  $\epsilon$  ensures numerical stability.

##### 2. RMSprop

RMSprop improves upon Adagrad by maintaining a decaying average of squared gradients, preventing the learning rate from becoming excessively small over time [74]. RMSprop maintains a decaying average of squared gradients,

$$G_t = \beta G_{t-1} + (1 - \beta) g_t^2, \quad \theta_{t+1} = \theta_t - \frac{\eta}{G_t + \epsilon} \nabla_{\theta} J(\theta_t) \quad (30)$$

where  $\beta$  is the decay rate.

### 3. Adaptive Moment Estimation (Adam)

Adam combines momentum and RMSprop to adaptively adjust learning rates based on first- and second-moment estimates of gradients [78]. Adam's stability and fast convergence make it one of the most widely used optimizers across various deep learning tasks. Adam combines momentum and RMSprop for adaptive learning rates:

$$M_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (31)$$

$$\widehat{M}_t = \frac{m_t}{1 - \beta_1^t}, \quad \widehat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (32)$$

$$\Theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\widehat{v}_t} + \epsilon} \widehat{m}_t \quad (33)$$

where  $m_t$  and  $v_t$  are the first and second moments, and  $\beta_1, \beta_2$  are decay rates.

### 4. Adadelta

Adadelta, an improved variant of Adagrad, addresses the issue of diminishing learning rates by employing a moving average of squared gradients instead of accumulating all past gradients [79]. It adapts the learning rate based on recent gradients rather than accumulating all past gradients. Mathematical Expression of Adadelta,

$$\Delta \theta_t = - \frac{\eta}{\sqrt{E[\Delta \theta]_{t-1}^2 + \epsilon}} \cdot \nabla_{\theta} J(\theta_t) \quad (34)$$

where  $E[\Delta \theta]_{t-1}$  is the moving average of squared gradients, and  $\epsilon$  is a small constant.

## 3.3 Classic CNN models

The following table (Table 3) provides a comparative overview of some classic CNN models, including their key features, accuracy, advantages, and suitability for various applications, offering a clear understanding of their evolution and practical use cases.

**Table 3** Overview and comparison of different CNN models

| Model                    | Year | Accuracy metrics<br>(Top-1/Top-5) | Parameters<br>(Million) | GFLOPs | Computational<br>costs (MACs/<br>Inference Time) | Key<br>features                                          | Improvements                         | Advantages                                 | Disad-<br>vantages                                   | Suit-<br>able<br>appli-<br>cations                            |
|--------------------------|------|-----------------------------------|-------------------------|--------|--------------------------------------------------|----------------------------------------------------------|--------------------------------------|--------------------------------------------|------------------------------------------------------|---------------------------------------------------------------|
| LeNet-5<br>[11]          | 1998 | ~99.0%/~99.05%<br>(MNIST)         | 0.06                    | 0.002  | ~0.001B<br>MACs/<1 ms                            | 5 layers,<br>small<br>filters,<br>pooling                | First practical<br>CNN               | Simple, fast<br>training                   | Limited<br>to small<br>datasets                      | Digit<br>recog-<br>nition,<br>basic clas-<br>sification       |
| AlexNet<br>[4]           | 2012 | 57.2%/84.6%<br>(ImageNet)         | 60                      | 1.43   | ~0.72B<br>MACs/~8.9 ms                           | ReLU<br>activation,<br>Drop-<br>out,<br>deeper<br>layers | GPU usage,<br>improved<br>accuracy   | High<br>accuracy for<br>ImageNet           | Overfit-<br>ting risk,<br>compu-<br>tational<br>cost | Image<br>clas-<br>sifica-<br>tion,<br>object<br>detection     |
| VGG [5,<br>80]           | 2014 | 71.5%/92.7%<br>(ImageNet)         | 138                     | 15.47  | ~15.5B<br>MACs/~15.6 ms                          | 16–19<br>layers,<br>small<br>filters                     | Improved<br>feature<br>extraction    | Simple<br>struc-<br>ture, high<br>accuracy | Large<br>model<br>size,<br>slow<br>training          | Image<br>clas-<br>sifica-<br>tion,<br>transfer<br>learning    |
| GoogleNet<br>[81]        | 2014 | 69.8%/93.3%<br>(ImageNet)         | 7                       | 1.5    | ~1.43B<br>MACs/~6.6 ms                           | Incep-<br>tion<br>mod-<br>ules, 22<br>layers             | Efficient<br>architecture            | Faster,<br>smaller size                    | Com-<br>plex to<br>imple-<br>ment                    | Object<br>recog-<br>nition,<br>real-<br>time<br>tasks         |
| ResNet<br>[82]           | 2015 | 76.2%/96.4%<br>(ImageNet)         | 25.6                    | 4.1    | ~4.1B<br>MACs/~9.8 ms                            | 152 lay-<br>ers, skip<br>connec-<br>tions                | Solves<br>vanishing<br>gradients     | Scalable for<br>very deep<br>networks      | Increased<br>training<br>time                        | Image<br>clas-<br>sifica-<br>tion,<br>deep<br>tasks           |
| DenseNet<br>[16, 83]     | 2017 | 74.9%/96.4%<br>(ImageNet)         | 8                       | 2.9    | ~2.9B<br>MACs/~10.5 ms                           | Dense<br>connec-<br>tions,<br>fewer<br>param-<br>eters   | Better feature<br>propagation        | Reduces<br>overfitting                     | High<br>memory<br>usage                              | Image<br>clas-<br>sifica-<br>tion,<br>medi-<br>cal<br>imaging |
| Xception<br>[7],<br>[26] | 2017 | 79.0%/94.5%<br>(ImageNet)         | 24.5                    | 4.7    | ~8.4B<br>MACs/~12 ms                             | Dep-<br>thwise<br>sepa-<br>rable<br>convolu-<br>tions    | Efficient,<br>reduced<br>computation | Light-<br>weight, fast<br>inference        | Slight<br>accuracy<br>drops                          | Image<br>analy-<br>sis,<br>transfer<br>learning               |
| MobileNet<br>[26]        | 2017 | 70.9%/89.9%<br>(ImageNet)         | 4.2                     | 0.57   | ~0.57B<br>MACs/~6.2 ms                           | Light-<br>weight,<br>efficient                           | Optimized for<br>edge devices        | Low compu-<br>tation, fast                 | Slightly<br>lower<br>accuracy                        | Mobile<br>vision,<br>IoT,<br>embed-<br>ded<br>systems         |

**Table 3** (continued)

| Model                            | Year | Accuracy metrics<br>(Top-1/Top-5) | Parameters<br>(Million) | GFLOPs | Computational<br>costs (MACs/<br>Inference Time) | Key<br>features                                      | Improvements                                 | Advantages                                        | Disad-<br>vantages                                      | Suit-<br>able<br>appli-<br>cations                             |
|----------------------------------|------|-----------------------------------|-------------------------|--------|--------------------------------------------------|------------------------------------------------------|----------------------------------------------|---------------------------------------------------|---------------------------------------------------------|----------------------------------------------------------------|
| NASNet<br>[84]                   | 2018 | 82.7%/96.2% (ImageNet)            | 88.9                    | 20.3   | ~ 23.8B<br>MACs/~ 20 ms                          | Auto-<br>mated<br>archi-<br>tecture<br>search        | Optimized<br>accuracy and<br>efficiency      | High<br>accuracy,<br>scalable                     | Compu-<br>tation-<br>ally<br>expen-<br>sive<br>search   | Image<br>clas-<br>sifica-<br>tion,<br>auto-<br>mated<br>ML     |
| Efficient-<br>Net<br>[6]         | 2019 | 77.1%/97.1% (ImageNet)            | 5.3                     | 0.39   | ~ 23.8B<br>MACs/~ 20 ms                          | Model<br>scaling<br>tech-<br>niques                  | High<br>accuracy<br>with fewer<br>parameters | Efficient,<br>lightweight                         | Com-<br>plex<br>scaling<br>strategy                     | Effi-<br>cient<br>image<br>classi-<br>fication                 |
| ViT<br>[20, 21]                  | 2020 | 81.0%/97.3% (ImageNet)            | 86                      | 55.4   | ~ 55.4B<br>MACs/~ 33 ms                          | Self-<br>atten-<br>tion,<br>patch-<br>based<br>input | Scales well to<br>large datasets             | High accu-<br>racy, avoids<br>convolution<br>bias | Requires<br>large data-<br>sets and<br>compute          | Image<br>clas-<br>sifica-<br>tion,<br>NLP<br>appli-<br>cations |
| Swin<br>Trans-<br>former<br>[21] | 2021 | 81.3%/98.7% (ImageNet)            | 88                      | 47     | ~ 4.5B<br>MACs/~ 7.1 ms                          | Shifted<br>window<br>attention                       | Improved<br>scalability<br>and efficiency    | High<br>accuracy,<br>hierarchical<br>design       | Compu-<br>tation-<br>ally<br>expen-<br>sive<br>training | Object<br>detection,<br>seg-<br>menta-<br>tion                 |

## 4 Key improvements in CNN

CNNs have evolved significantly to enhance efficiency, scalability, and feature representation, addressing key challenges in deep learning. Key improvements in convolutional layers, pooling techniques, and activation functions have enhanced CNN performance, enabling their application across diverse domains. These innovations optimize computational efficiency, adapt to complex tasks, and improve model generalization, making CNNs more effective and versatile for modern deep learning applications.

### 4.1 Architectural improvements

#### 4.1.1 Enhanced convolutional layer techniques

Convolutional layer techniques have evolved significantly to address the challenges of computational efficiency, scalability, and feature representation in deep learning. By introducing novel methods to refine the way features are extracted and processed, these techniques enhance the performance of neural networks across various domains. They are designed to improve efficiency, reduce complexity, and adapt to the diverse requirements of modern applications, making deep learning models more versatile and effective.

**4.1.1.1 Depthwise separable convolution** A network architecture technique called Depthwise Separable Convolution divides a convolution operation into two components: pointwise convolution, which expands the feature map's dimension by combining data from various channels, and depthwise convolution, which works on individual input channels. This approach reduces the number of parameters and computation required compared to standard convolution [1]. This approach efficiently captures spatial features and learns cross-channel relationships while significantly reducing computational complexity. The total cost is reduced from  $K^2 \times M \times N \times D$  in standard convolution to  $K^2 \times D + D \times P$  in depthwise separable convolution. It is particularly suited for mobile

devices, real-time tasks, and deployment on Field-Programmable Gate Array (FPGA), ASICs, and embedded systems for robotics and computer vision [7].

**4.1.1.2 Dilated convolution** Dilated Convolution, or Atrous Convolution [85], extends standard CNNs by introducing a dilation rate ( $l$ ) to expand the receptive field without increasing parameters or computation. It achieves this by spacing out kernel elements with zeros, allowing the network to capture broader contextual information [86]. For a 1-D signal  $F$  and kernel  $k$  of size  $r$ , the  $l$ -dilated convolution is defined as:

$$(F \cdot k)_t = \sum_{\tau} k_{\tau} F_{t-\tau} \quad (35)$$

This extends naturally to 2-D. The receptive field grows exponentially with dilation: factors 1, 2, 4 result in field sizes  $3 \times 3$ ,  $7 \times 7$ , and  $15 \times 15$  respectively [86]. In applications requiring broad receptive fields, like speech synthesis, machine translation, picture segmentation, and speech recognition, Dilated CNNs perform exceptionally well, according to the authors of.

**4.1.1.3 Inception module** Initially introduced as GoogLeNet, The Inception module, introduced by Szegedy et al. [87] designed for image recognition tasks. It builds on the Network in Network (NIN) approach and enhances image recognition by using multiple parallel convolutional filters of varying sizes ( $1 \times 1$ ,  $3 \times 3$ ,  $5 \times 5$ ) and a max pooling operation. This allows the network to capture features at different scales and levels of abstraction. The outputs of these pathways are concatenated to form a rich feature map. This design allows for deeper and wider CNNs without increasing computational complexity, reducing network parameters to 5 million—much fewer than AlexNet (60 million) and ZFNet (75 million). In a later paper [88], they suggest gradually reducing representation size and using spatial aggregation to maintain performance with modest computation. Inspired by ResNet, Inception-V4 combines Inception with shortcut connections, enhancing training speed and achieving a 3.08% top 5 error rate on the ILSVRC 2012 validation set.

**4.1.1.4 Transpose convolution** Transposed convolution, also known as fractionally strided convolution [89], is the reverse of traditional convolution. In contrast to ordinary convolution, which links several input activations to one output, deconvolution links one input activation to several output activations. It works by first upsampling the input using a stride and padding and then applying a convolution operation to the upsampled input. Transpose convolution is widely applied in tasks such as image reconstruction, semantic segmentation, visual question answering, and super-resolution [86].

**4.1.1.5 Network in network** Network in Network (NIN), proposed by Lin et al. [90], replaces the linear filter in traditional convolutional layers with a micro network, such as a multilayer perceptron convolution (MLPconv) layer, enabling the approximation of more abstract representations of latent concepts. The NIN architecture is formed by stacking these micro networks. In a standard convolutional layer (with a nonlinear activation function), the feature map is computed as:

$$a_{i,j,k} = \max \left( w_k^T x_{i,j} + b_k, 0 \right) \quad (36)$$

where  $a_{i,j,k}$  is the activation of the  $k$ -th feature map at position  $(i,j)$ ,  $x_{i,j}$  is the input patch at position  $(i,j)$ , and  $w_k$  and  $b_k$  are the weight vector and bias term for the  $k$ -th filter. In contrast, the mlpconv layer operates as:

$$a_{i,j,kn}^n = \max \left( w_{kn}^T a_{i,j,:}^{n-1} + b_{kn}, 0 \right) \quad (37)$$

where  $n \in [1, N]$  and  $N$  is the number of layers in the mlpconv, with  $a_0 = x_{i,j}$ . The mlpconv layer applies  $1 \times 1$  convolutions after the traditional convolution, which acts as cross-channel parametric pooling followed by ReLU activation. Thus, the mlp conv layer can be seen as cascading cross-channel pooling operations on the standard convolutional output.

## 4.1.2 Advances in pooling layers

Pooling methods play a crucial role in deep learning by reducing the spatial dimensions of feature maps while preserving key information. Over the years, several innovative pooling techniques have been introduced, each

**Table 4** Summary of advanced pooling techniques

| Method              | Key parameters                                   | Primary advantages                             | Main disadvantages                               | Original reference |
|---------------------|--------------------------------------------------|------------------------------------------------|--------------------------------------------------|--------------------|
| Average pooling     | $2 \times 2/3 \times 3$ pool, stride 1–2         | Smooth gradients, noise suppression            | Information dilution, poor sparse features       | [91]               |
| Lp pooling          | $p \in [1, 10]$                                  | Biologically inspired, flexible                | Computational overhead, hyperparameter sensitive | [92]               |
| Mixed pooling       | Mixing weight $w \in [0, 1]$                     | Regularization, combines avg/max strengths     | 5–15% overhead, parameter sensitive              | [31]               |
| Stochastic pooling  | $3 \times 3$ regions optimal                     | Superior regularization, model averaging       | Requires ReLU, training stochasticity            | [32]               |
| Spectral pooling    | Frequency cutoff, pyramid levels                 | Information preservation, avoids aliasing      | Complex arithmetic, $O(HW \log(HW))$             | [34]               |
| SPP                 | Pyramid $\{4 \times 4, 2 \times 2, 1 \times 1\}$ | Variable inputs, 24–102× speedup               | Placement constraints, training complexity       | [35]               |
| Super-pixel pooling | SLIC compactness, superpixel count               | Boundary/edge preservation, semantic coherence | SLIC preprocessing, irregular parallelization    | [93]               |

**Table 5** Summary of activation function [94]

| Name               | Monotonic derivative | Range                | Order of continuity |
|--------------------|----------------------|----------------------|---------------------|
| Sigmoid            | Yes                  | $(-\infty, +\infty)$ | $C^\infty$          |
| Hyperbolic Tangent | Yes                  | $(-1, 1)$            | $C^\infty$          |
| ReLU               | Yes                  | $[0, \infty)$        | $C^0$               |
| Leaky ReLU         | Yes                  | $(-\infty, +\infty)$ | $C^0$               |
| PReLU              | Yes                  | $(-\infty, +\infty)$ | $C^0$               |
| Swish              | No                   | $(-\infty, +\infty)$ | $C^\infty$          |
| Mish               | No                   | $[-0.31, +\infty)$   | $C^\infty$          |

**Table 6** Characteristics of common activation functions. [94, 95]

| Activation function | Diminishing gradient | Limited non-linearity | Optimization difficulty | Lack of adaptability | Computational inefficiency |
|---------------------|----------------------|-----------------------|-------------------------|----------------------|----------------------------|
| Sigmoid             | Yes                  | No                    | Yes                     | Yes                  | Yes                        |
| Tanh                | Yes                  | No                    | Partial                 | Yes                  | Yes                        |
| ReLU                | Partial              | Yes                   | Partial                 | Yes                  | No                         |
| ELU                 | No                   | Partial               | No                      | Yes                  | Partial                    |
| APL                 | No                   | Partial               | No                      | No                   | No                         |
| Swish               | No                   | Partial               | No                      | No                   | Partial                    |

with unique approaches to balance information retention, computational efficiency, and generalization. Table 4 below summarizes key pooling methods, their descriptions, origins, and benefits.

### 4.1.3 Advanced activation functions

In a paper [94], the authors present tables summarizing the differences and characteristics of common activation functions (shown in Tables 5 and 6) for easier interpretation.

## 4.2 Training techniques

### 4.2.1 Advanced loss functions

In order to train CNNs for both regression and classification tasks, loss functions are essential. They measure the difference between predicted and actual values, guiding the model toward better predictions. Various loss functions

**Table 7** Comparison between different types of advanced loss functions

| Loss function        | Key features and parameters                     | Primary advantages                                 | Disadvantages                                    | References |
|----------------------|-------------------------------------------------|----------------------------------------------------|--------------------------------------------------|------------|
| MAE                  | Linear penalty, no hyperparameters              | Robust to outliers, stable gradients               | Non-differentiable at zero, slower convergence   | [97]       |
| MSE                  | L2 penalty, quadratic                           | Smooth optimization, fast convergence              | Extremely sensitive to outliers, scale-dependent | [98]       |
| Cross-entropy        | Softmax normalization, temperature T            | Probabilistic interpretation, smooth gradients     | Sensitive to class imbalance, label noise        | [99]       |
| Hinge loss           | Convex, non-differentiable                      | Maximum margin, sparse solutions (SVMs)            | No probabilistic output, primarily binary        | [64]       |
| Softmax loss         | Combined with cross-entropy, temperature T      | Direct probabilities, universal classification     | Computational bottleneck for large vocabularies  | [100]      |
| Large margin Softmax | Margin $m$ (2–4), angular enforcement           | Enhanced discrimination, superior face recognition | Difficult optimization, hyperparameter sensitive | [101]      |
| Contrastive loss     | Margin $m$ ( $\sim 1.0$ ), Euclidean distance   | Simple pairwise learning, stable training          | $O(N^2)$ scaling, greedy pair collapse           | [67];      |
| Triplet loss         | Margin $\alpha$ (0.2–1.0), hard negative mining | Relative distance learning, prevents collapse      | $O(N^3)$ scaling, requires sophisticated mining  | [69]       |
| Double margin loss   | Dual margins $m_1, m_2$ , normalized distance   | Faster convergence, dual margin control            | Limited to pairwise, hyperparameter sensitive    | [102]      |

are tailored for specific tasks, with their strengths and weaknesses depending on the dataset and the nature of the problem. In the paper [86, 96] the authors discussed about these loss functions. Following Table 7 summarizes the most commonly used loss functions, their working principles, and applications in real-world datasets.

#### 4.2.2 Advanced optimization methods

In CNNs, optimizing nonconvex functions requires significant computational resources. To efficiently minimize the loss function and obtain optimal network parameters, optimizers like Momentum, RMSprop, and Adam are commonly used during training. These methods reduce computation time while improving convergence. The advantages and disadvantages of these methods were discussed in [86, 96], here is a concise explanation of the findings in Table 8.

#### 4.2.3 Advanced regularization techniques

Enhancing model performance and generalization while reducing overfitting is key in machine learning. Techniques like regularization, Dropout, normalization, and data augmentation address these challenges. The Table 9 below provides a concise overview of these methods with their descriptions and benefits.

**4.2.3.1 CNN-RNN hybrids** Hybrid architectures, such as CNN-RNN models (Fig. 11), effectively combine spatial and temporal feature extraction. These models enhance performance in tasks such as construction duration estimation, music genre classification, image captioning, sentiment analysis, and stock market prediction by leveraging CNNs for feature extraction and RNNs for sequential dependencies. For instance, a Spatio-Temporal Attention CNN-RNN model effectively addresses uncertainties in construction projects by integrating attention mechanisms, while a hybrid CNN-LSTM architecture excels in music genre classification by capturing both spatial features and temporal dependencies, achieving superior accuracy in complex genres [103, 104]. Additionally, the integration of CNNs and Long Short-Term Memory networks (LSTMs) in image captioning has demonstrated improved contextual relevance and fluency in generated captions [105]. In sentiment analysis, the hybrid model significantly enhances classification accuracy by capturing intricate data patterns [106]. Additionally, CNN-RNN hybrids have demonstrated improved performance in stock market prediction by capturing short-term and long-term dependencies more effectively than standalone RNNs.

A key challenge lies in balancing model complexity and training time, as sequential dependencies can significantly increase computational load. To mitigate overfitting and vanishing gradients, techniques such as dropout,

**Table 8** Comparison between different types of optimization methods

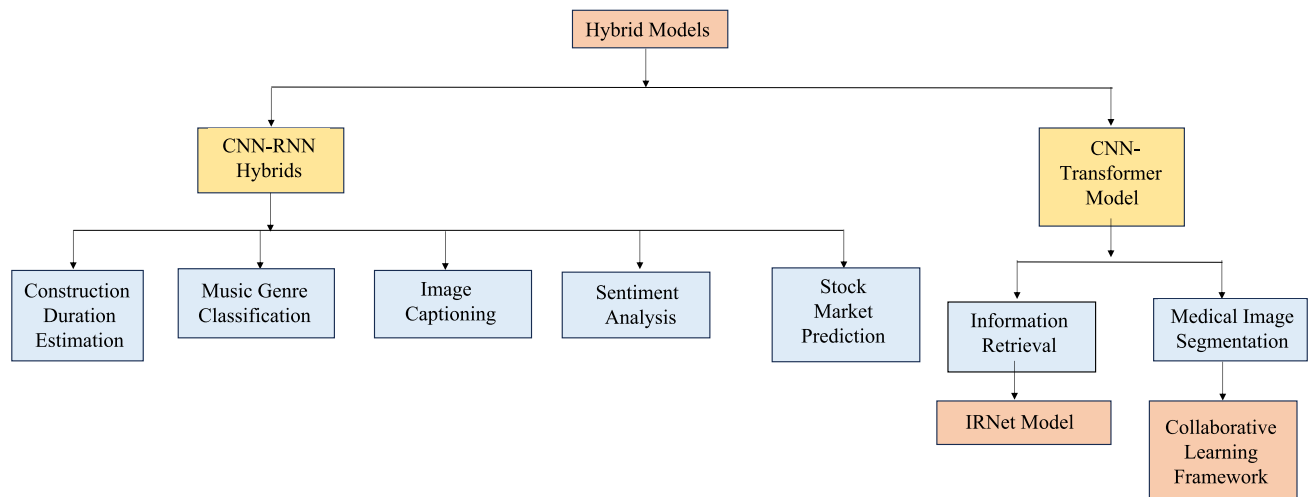
| Method name                | Advantages                                                                                                                                     | Disadvantages                                                                                                             |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| Data Augmentation          | Reduces overfitting, Enhances generalization, Effective for limited data                                                                       | Greedy selection is computationally expensive, Risk of irrelevant transformations                                         |
| Weight Initialization      | Improves convergence speed, Reduces gradient issues                                                                                            | Poor initialization leads to vanishing/exploding gradients                                                                |
| Xavier Initialization      | Automatically adjusts scale, Works well for tanh/sigmoid activations                                                                           | Not suitable for ReLU-based networks                                                                                      |
| He Initialization          | Robust for ReLU-based deep networks, Reduces gradient problems                                                                                 | May not generalize to other nonlinearities                                                                                |
| Orthonormal Initialization | Efficient decorrelation of layers, Applied with BN                                                                                             | More complex than standard initialization                                                                                 |
| SGD                        | Computationally efficient for large datasets, mini-batches stabilize convergence, widely used in deep learning                                 | Sensitive to learning rate selection, may not always converge, slower in non-convex optimization landscapes               |
| Momentum                   | Faster convergence, Smooths updates by incorporating gradient history                                                                          | Poorly tuned momentum can overshoot minima., Still sensitive to learning rate                                             |
| Nesterov Momentum          | Prevents overshooting, Achieves better performance compared to classical momentum                                                              | More computationally expensive than standard momentum, requires careful hyperparameter tuning                             |
| Parallelized SGD           | Accelerates training with multiple GPUs/CPU, reduces delays caused by slow nodes                                                               | May suffer from stale gradients in asynchronous setups, Requires careful implementation for consistency                   |
| BN                         | Mitigates internal covariate shift, allows higher learning rates without divergence, improves gradient flow, and enhances model regularization | Introduces computational overhead, less effective for very small mini-batches, complex to implement in some architectures |

**Table 9** Advanced machine learning regularization techniques

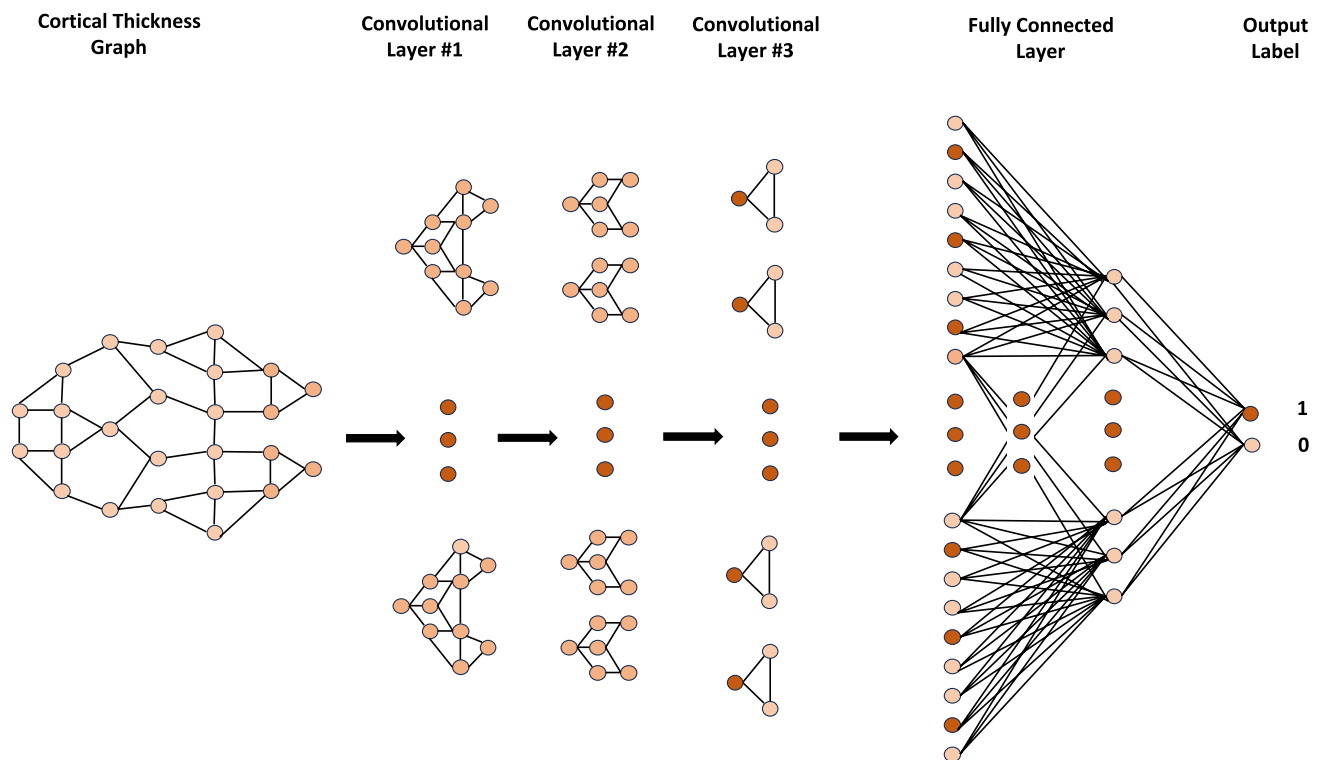
| Method              | Key parameters                                            | Advantages                                                               | Disadvantages                                               |
|---------------------|-----------------------------------------------------------|--------------------------------------------------------------------------|-------------------------------------------------------------|
| L2 Regularization   | Penalty $\lambda$ (0.001–10), circular constraints        | Handles multicollinearity, stable solutions, fast convergence            | No feature selection, requires standardization              |
| Dropout             | Probability $p$ (0.2–0.5), random neuron deactivation     | Prevents co-adaptation, reduces overfitting, easy implementation         | Different train/test behavior, can slow convergence         |
| DropConnect         | Weight mask probability $p$ (0.1–0.3)                     | Fine-grained regularization, better parameter efficiency                 | Higher computational overhead, complex implementation       |
| Batch Normalization | Learnable $\gamma$ , $\beta$ parameters, batch statistics | Accelerates training, enables higher learning rates, stabilizes networks | Batch size dependency, memory overhead                      |
| Data Augmentation   | Transform intensity, application probability              | Increases dataset size, improves generalization, cost-effective          | May introduce unrealistic samples, hyperparameter sensitive |

layer normalization, and attention mechanisms are incorporated. Data augmentation and early stopping are also used to prevent overfitting in limited datasets [66].

**4.2.3.2 CNN-transformer models** CNN-Transformer models represent a significant advancement in various domains, particularly in information retrieval and medical image segmentation. These hybrid architectures leverage the strengths of CNNs and Transformers, addressing their individual limitations. For instance, the IRNet model integrates local and global information through a dual-stream architecture, enhancing retrieval efficiency by improving recall and precision by approximately 7% and 8%, respectively [107]. Similarly, CNN-Transformer architectures in medical image segmentation enable bi-directional knowledge transfer between local and global features, improving segmentation accuracy on benchmark datasets [108] performance on benchmark datasets. Additionally, these models have shown promise in computer vision tasks, capturing long-range dependencies and contextual information effectively [109]. Innovations such as lightweight CNN-Transformer models for solving traveling salesman problems demonstrate their versatility and efficiency in handling complex tasks while reduc-



**Fig. 11** Hybrid architecture of CNN combining CNN–RNN and CNN–Transformer models with their respective applications



**Fig. 12** Graph-CNN Architecture on Cortical Thickness Graph

ing computational cost [110]. Furthermore, in medical diagnostics, a hybrid model for liver cancer detection achieved an AUC of 0.8237, showcasing its potential for clinical applications [111].

One of the main challenges is the high computational cost of Transformer layers. Solutions include using lightweight variants like MobileViT, hierarchical transformers, or patch-based processing to reduce the attention computation load [112]. Another issue is the mismatch in inductive biases: CNNs capture local patterns effectively, whereas Transformers are more flexible but require more data to generalize well. Combining them mitigates these weaknesses.

**4.2.3.3 Graph-CNNs in emerging architectures** Graph Convolutional Neural Networks (Graph-CNNs) extend the traditional convolution operation from regular Euclidean domains (like 2D images) to non-Euclidean domains such as graphs [113]. This is particularly beneficial for data that naturally resides in graph form, such as social

networks, molecular structures, traffic systems, and, importantly, 3D point clouds [114]. In these applications, relationships between data points are better expressed using nodes and edges rather than pixels on a grid.

A representative example of a Graph-CNN architecture is shown in Fig. 12. In this illustration, a cortical thickness graph is used as input—each node represents a region (or feature point) in the cortical surface, and the edges encode spatial relationships among regions. The graph convolution layers operate by aggregating features from each node's local neighborhood [115]. This enables learning from the underlying structure and relationships within the graph.

As shown in Fig. 12, the process begins with a graph-structured input fed into a series of convolutional layers. Each graph convolution layer performs message passing, where the features of each node are updated by aggregating features from its neighbors. This is repeated across layers to capture increasingly complex and abstract representations of the graph's topology and features [116]. Eventually, the features are flattened and passed through a fully connected (FC) layer, followed by a classifier that outputs the final prediction (e.g., class label 0 or 1). This pipeline is particularly useful in applications like medical imaging, where brain regions (e.g., cortical thickness) can be represented as graph nodes and analyzed structurally [117].

Graph-CNNs have gained substantial traction in 3D point cloud processing, where the input data is a set of 3D points that lack an inherent order or grid structure. Initial models such as PointNet and PointNet++ revolutionized this field by directly operating on raw point cloud data [118]. These models used shared Multi-Layer Perceptrons to independently process each point and employed symmetric functions (like max pooling) to aggregate global features. However, they struggled with capturing local geometric context.

To address this limitation, Dynamic Graph CNN (DGCNN) introduced the EdgeConv operation, which dynamically constructs a graph from point clouds in the feature space rather than fixed input space [119]. This allows the network to learn local geometric relationships at each layer adaptively. Points are treated as nodes, and edges are formed using k-nearest neighbors (k-NN), enabling more expressive local neighborhood learning.

Further advancements include Point-GCN, which employs spectral graph convolutions to extract spatially localized features [120], and RandLA-Net, which scales to large point clouds by using random sampling and hierarchical local feature aggregation [121]. These architectures excel in tasks like 3D object classification, semantic segmentation, and scene understanding.

To implement Graph-CNNs in point cloud applications, one typically constructs a graph using k-NN to define neighborhood connections among points. Graph convolution operations are then applied using either spectral methods (e.g., Laplacian-based filters) or spatial methods (e.g., neighbor aggregation) [122]. Tools like PyTorch Geometric and Deep Graph Library offer robust implementations for both approaches, enabling researchers to efficiently build and test custom graph-based neural networks [123]. By combining the flexibility of graph representation with the learning power of CNNs, Graph-CNNs serve as a versatile tool for learning from structured and unstructured non-Euclidean data across various domains.

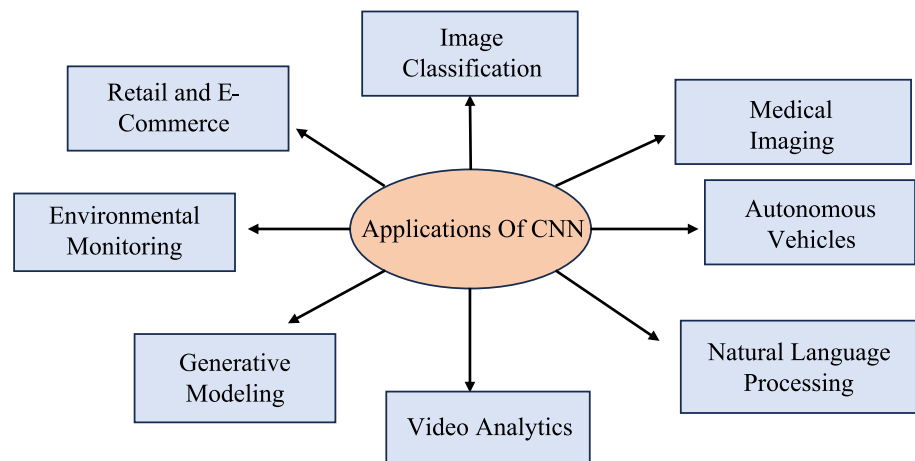
## 5 Diverse and specialized applications of CNNs

CNNs have been pivotal in driving advancements across diverse domains by leveraging their ability to learn hierarchical representations of data. Their versatile architecture and adaptability have made them an essential tool in solving complex problems in fields ranging from healthcare to entertainment. This section explores some crucial applications (Fig. 13) of CNNs, highlighting their significance and transformative impact.

### 5.1 Medical imaging

CNNs have transformed medical imaging by making it possible to precisely identify, categorize, and segment anomalies. They play a vital role in detecting life-threatening conditions such as lung cancer, brain tumors, and cardiovascular diseases [124]. For example, CNNs are employed in analyzing mammograms to detect breast cancer with accuracy comparable to expert radiologists [125]. U-Net, a widely used CNN architecture, specializes

**Fig. 13** Diverse and Specialized Applications of CNNs across multiple domains



in medical image segmentation, making it indispensable in surgical planning and radiotherapy [16]. Moreover, CNNs enhance retinal disease diagnosis, detecting diabetic retinopathy and age-related macular degeneration with minimal human intervention [126]. In pathology, they assist in identifying cancerous tissues in histopathological slides, automating processes that were previously labor-intensive [127]. CNN-based methods have also shown strong performance in colorectal cancer screening, supporting tasks such as detection, localization, segmentation, and classification with high accuracy and robustness [128]. A lightweight CNN-based model, PD-CNN-PCC-EELM, achieved high accuracy in classifying 27 gastrointestinal diseases, with explainable AI techniques enhancing the interpretability of its diagnostic decisions [129]. Another CNN-based model, PSE-CNN-PCA-DELM, further improved Gastrointestinal disease detection accuracy using a tri-stage architecture combined with Explainable Artificial Intelligence (XAI) tools like Grad-CAM and SHAP for transparent diagnosis [130]. A customized CNN architecture, SPCNN, achieved 99.37% accuracy in malaria parasite detection, with Grad-CAM and SHAP enhancing its interpretability [131]. A Multi-Scale CNN framework achieved 96.05% accuracy across seven lung disease classes, showcasing its strength in complex multi-class medical diagnosis tasks [132]. Recent studies have also demonstrated the effectiveness of CNNs in pneumonia detection using chest X-ray images, achieving high diagnostic accuracy through deep learning-based models [133]. Similarly, CNN architectures have been applied to skin cancer detection and segmentation tasks, improving early diagnosis and classification with reliable performance across various lesion types [134]. An interpretable deep learning model has also been proposed for tuberculosis detection from X-ray images, showing promising accuracy while emphasizing model transparency and trustworthiness [135]. Furthermore, collaborative federated learning frameworks based on CNNs have enabled privacy-preserving training across multiple institutions, significantly advancing lung and colon cancer classification without compromising patient data security [136, 137].

## 5.2 Autonomous vehicles

The deployment of CNNs in autonomous vehicles has transformed the transportation industry by enabling vehicles to perceive and navigate their environment safely. Real-time object detection models like YOLO and SSD (Single Shot Multibox Detector) allow vehicles to identify pedestrians, cyclists, and other vehicles [138, 139]. Semantic segmentation networks, such as DeepLab and SegNet, enable lane detection, traffic signal recognition, and road condition assessment [140, 141]. Advanced autonomous systems fuse CNN-based visual processing with LiDAR and radar data, generating accurate 3D environmental maps for robust navigation in varying conditions, including low visibility due to heavy rain or fog [142]. Tesla's Autopilot and Waymo's self-driving systems extensively utilize CNNs for adaptive cruise control and obstacle avoidance [143].

### 5.3 Natural language processing (NLP)

Although CNNs are primarily known for image processing, they have significantly impacted NLP. They excel in text classification tasks, including spam detection, sentiment analysis, and news categorization [15]. CNNs capture n-gram features effectively, identifying patterns within phrases and sentences. For instance, in customer feedback analysis, CNNs determine sentiment polarity, helping businesses improve their services [144]. CNNs also contribute to machine translation, converting text between languages by analyzing syntactic and semantic structures [145]. When combined with recurrent networks or transformers, CNNs enhance performance in complex tasks like summarization, question answering, and contextual word embeddings [146].

### 5.4 Video analytics

Video analytics leverages CNNs for a broad spectrum of applications, from real-time surveillance to entertainment. Spatiotemporal models such as C3D (3D Convolutional Networks) capture motion dynamics, enabling precise action recognition in videos [147]. For instance, CNNs are used in monitoring live sports events to provide instant replays and player performance statistics [71]. Surveillance systems equipped with CNNs detect anomalies such as loitering or unauthorized access, enhancing security in public spaces [148]. In cinematic applications, CNNs facilitate video editing by identifying scene transitions and generating visual effects [149].

### 5.5 Generative modeling

CNNs are integral to generative modeling techniques, creating new data instances from existing datasets. GANs (Generative Adversarial Networks) and VAEs are among the most prominent models leveraging CNNs [72]. GANs and VAEs are prominent generative models utilizing CNNs. GANs generate photorealistic images for applications in gaming, advertising, and virtual reality [150] while VAEs are widely used in anomaly detection, data augmentation, and synthetic image generation by learning latent representations [151]. In healthcare, generative models synthesize medical images to augment training datasets, reducing biases in machine learning models [152]. CNNs also support artistic endeavors, including style transfer, where an image adopts the artistic features of another, and super-resolution techniques that enhance image quality.

### 5.6 Environmental monitoring

Environmental science has greatly benefited from CNNs, especially in analyzing satellite and drone imagery for sustainability efforts. For example, CNNs monitor deforestation by detecting changes in forest cover over time [153]. In agriculture, precision farming applications use CNNs to identify nutrient deficiencies, diseases, and pest infestations in crops, optimizing resource usage [154]. A lightweight PDSCNN model achieved 94.22% accuracy in plant disease classification, demonstrating rapid inference times suitable for deployment on edge devices like Jetson Nano [155]. A novel three-stage waste classification system was proposed, incorporating a parallel lightweight depth-wise separable convolutional neural network (DP-CNN) to enhance classification accuracy and efficiency [156]. Climate researchers utilize CNNs to model weather patterns and predict natural disasters such as hurricanes and floods, mitigating their impact [157]. Advanced CNN applications map biodiversity by analyzing animal populations in remote regions using aerial images, aiding conservation efforts [158]. The integration of CNNs in these areas supports sustainable development and environmental preservation on a global scale.

### 5.7 Retail and E-commerce

Retail and e-commerce sectors have embraced CNNs to enhance customer experience and operational efficiency. Visual search engines powered by CNNs enable users to find products by uploading images, streamlining the

shopping process. Inventory management systems leverage CNNs for object detection, ensuring accurate stock monitoring and replenishment [159]. Virtual try-on applications simulate clothing or accessories on customers' images, bridging the gap between online and in-store shopping experiences [160]. Personalized recommendation engines use CNNs to analyze user preferences, boosting customer engagement and sales [161]. Moreover, CNNs assist in fraud detection by identifying suspicious patterns in online transactions, enhancing security [60].

## 5.8 Image classification

Large-scale image categorization had its breakthrough in 2012. In ILSVRC 2012, Krizhevsky et al. [162] developed AlexNet and achieved the highest performance. Several works have significantly improved classification accuracy after AlexNet's success by either increasing the network depth [81] or decreasing the filter size [28]. In order to acquire fine-grained features for subclass recognition, Wang et al. [163] construct a tree structure. A training technique that expands a network both hierarchically and gradually is put forth. In this approach, classes are self-organized into various levels and grouped based on commonalities.

Another quickly expanding area of image classification is subcategory classification. Some fine-grained age datasets are already available, such as those for cars, dogs, birds, and plants [164]. In general, localizing significant portions of objects and expressing their looks discriminatively can increase accuracy. In the process, Branson et al. [165] provide a technique that extracts CNN features from several pose-normalized regions and finds portions. A compact posture normalization space is learned using part annotation information. In order to increase classification accuracy, they additionally construct a model that combines higher-level feature layers with unaligned picture features and lower-level feature layers with pose-normalized extraction procedures.

## 5.9 Scene labeling

Scene labeling, or semantic segmentation, assigns a semantic class (e.g., road, water, sky) to each pixel in an image, enabling precise scene understanding in applications such as autonomous driving and remote sensing [166]. The class likelihood of pixels is directly modeled by CNNs from local picture patches. To distinguish the intricacies of local visuals, they can learn powerful features and classifiers. By using picture patch sampling to train the parametric CNNs, significantly reduced the training time. However, they discover that local ambiguity issues plague patch-based CNNs. In order to solve it, Shuai et al. [167] integrate global ideas. RNNs are used by Shuai et al. [168] to simulate contextual dependencies among CNN-derived visual features, significantly improving labeling performance.

For semantic segmentation tasks, researchers have used deep CNNs that have already been trained. Using ConvNet and AlexNet, Mostajabi et al. [169] combine local, neighboring, and global information to create zoom-out features for increased accuracy. Fully Convolutional Networks (FCNs), as proposed by Long et al. [127], create dense label maps by repurposing ImageNet-trained models and incorporating deconvolution layers to improve resolution. In order to improve segmentation results, Chen et al. [170] combine deep CNN outputs with fully connected Conditional Random Fields (CRFs) to overcome border misalignment difficulties.

### 5.9.1 Object detection

One of the main problems in computer vision has always been object detection [171]. Prior to 2012, inadequate training data and low processing power impeded advancements in CNN-based detection. However, interest in this field was rekindled [172] by CNNs' ground-breaking success in the 2012 ImageNet competition [162]. Methods that use object suggestions have recently drawn a lot of interest and are now a common area of study.

One well-known CNN-based object detection method is Region-based CNN (R-CNN). It relies Rich feature on Selective Search [173] to generate approximately 2000 candidate regions that may contain objects. These regions are resized to a uniform dimension, and a pre-trained CNN extracts features from each. A binary SVM

classifier is then applied to identify and detect objects. Recently, YOLO and SSD have introduced efficient single-stage object detection frameworks. By treating detection as a regression job, YOLO allows end-to-end optimization by predicting bounding boxes and class probabilities in a single network pass. SSD improves accuracy by using default boxes of varying scales and aspect ratios across feature maps.

### 5.9.2 Text detection

Text detection and recognition have greatly advanced with the use of CNNs. Early methods used CNNs to differentiate text from non-text patches, often combining them with Maximally Stable Extremal Regions to refine candidate regions and separate cluttered components. Beyond localization, CNNs were also applied to detect the presence of text without pinpointing its exact location [174]. For recognition, models like those by Goodfellow et al. [175] used CNNs to predict sequential characters, while Jaderberg et al. [176] introduced a CRF-like CNN for character sequences without relying on lexicons. Recent approaches combine CNNs with LSTMs to capture sequential dependencies for improved recognition. End-to-end systems integrate detection and recognition tasks, with CNNs performing text detection, classification, and bounding box refinement [177]. These methods demonstrate CNNs' ability to handle both text detection and recognition efficiently.

## 6 Future prospects

### 6.1 Security of CNNs

CNNs are integral to various critical applications, including security identification systems [178], medical image analysis [179], traffic sign recognition [180] and license plate recognition [181]. These applications are inherently connected to safety and security, necessitating robust measures to ensure the integrity and reliability of CNN models. However, there are serious risks to CNNs, which can be broadly divided into adversarial attacks and data poisoning.

#### 6.1.1 Data poisoning

Data poisoning occurs when noise or perturbations are introduced into training data during the learning phase, often remaining imperceptible at the image level. While the training process appears unaffected, the corrupted data compromises model performance during testing. In some instances, such as backdoor attacks, fine-tuned noise deliberately misleads the model into incorrect classifications of specific targets. For instance, Yang et al. [182] demonstrated the use of perturbation masks embedded into training samples as backdoors, enabling deceptive behavior without disrupting normal functionality. To address this issue, Branson et al. [165] proposed a combination of fine-tuning and pruning, effectively suppressing or eliminating the effects of backdoors.

#### 6.1.2 Adversarial attacks

Adversarial attacks involve adding subtle, imperceptible noise to input images, which causes CNN models to misclassify them. Xie et al. [183] attributed CNNs' susceptibility to adversarial attacks to their high-dimensional decision boundaries and locally linear behavior in activation functions like ReLU and Maxout. They introduced a fast gradient-based approach to generate adversarial examples, highlighting CNN models' susceptibility to such perturbations. In the paper [184], the authors suggested three key defense strategies: (1) employing adversarial examples during training to enhance robustness, (2) modifying network architectures to tolerate noise better, and (3) incorporating auxiliary networks to safeguard against adversarial attacks.

## 6.2 CNN and advances with CapsNets

While CNNs power a wide range of applications, from basic classifiers to autonomous vehicles, they are not without limitations. CNNs are insensitive to minor image changes like rotation, scaling, and symmetry [185]. For example, when trained on an image, CNNs may fail to recognize rotated or transformed versions that humans easily identify. Addressing this requires vast amounts of diverse data, which is often difficult to obtain. Data augmentation offers some improvement but remains limited.

CapsNets were introduced by Geoffrey Hinton in Dynamic Routing Between Capsules [10] to mitigate spatial information loss in CNNs. These networks use capsules—vector-based representations encoding an object’s existence and pose—forming tree-like structures of sparsely activated neurons. CapsNets recognize orientation and spatial relationships, with advancements like the stacked capsule autoencoder [186], which integrates top-down attention for pose and part recognition. While CapsNets have shown promise, they struggle with complex datasets like CIFAR-100 and ImageNet, highlighting areas for future improvement.

## 6.3 Hardware implementation

The complexity of neural networks increases the amount of computing power needed for both inference and training. Alternative hardware, such as GPUs and FPGAs, are being used more and more, even if CPU performance has plateaued. With the introduction of GPU-based solutions by NVIDIA, such as libraries like cuBLAS and cuDNN and architectures like Turing and Ampere, GPU clusters are now in control of deep learning systems.

With their flexible architecture, FPGAs provide an alternative for model-specific optimization, unlike fixed architectures like CPUs and GPUs. Early FPGA-based acceleration schemes, particularly convolutional kernel-based designs, faced limitations. Varying convolution kernel sizes (e.g., AlexNet’s transition from  $11 \times 11$  to  $5 \times 5$  and  $3 \times 3$ ) led to inefficient parallelism and resource waste. Emerging kernels like  $1 \times 1$  and asymmetric designs posed challenges for parallelism in fixed architectures, sometimes leading to inefficient resource utilization. However, modern FPGA and GPU architectures leverage optimized parallel execution, including SIMD and specialized tensor cores, to maintain high throughput across different kernel sizes. Among FPGA acceleration schemes, early designs relied on low-frequency CPUs (softcore or embedded ARM Cortex A9) for control functions, limiting them to embedded systems rather than cloud-scale acceleration. However, cloud-based FPGA solutions, such as Microsoft’s Catapult project, demonstrated nearly  $2 \times$  acceleration for Bing ranking in data centers.

Experiments show that FPGAs can offer superior power efficiency compared to CPUs and, in some cases, GPUs, particularly for low-latency inference workloads. With advancements in FPGA integration and clock frequency, they are poised to play a more significant role in the next wave of deep learning hardware innovation.

## 6.4 Explainability and interpretability of CNNs

CNNs have proven to perform exceptionally well in a variety of fields, such as natural language processing, picture classification, and medical diagnostics. Their “black box” character, however, poses questions regarding transparency, particularly in crucial applications that demand responsibility and trust. The difficulties in comprehending CNNs, methods for making them easier to understand, and their significance in delicate applications are all covered in this section.

### 6.4.1 Interpretability challenges and future research directions in deep CNNs

Deep CNNs possess complex architectures comprising multiple layers and millions of parameters, making their inner workings challenging to interpret. Key challenges include:

**6.4.1.1 Opacity of internal representations** CNNs transform raw input data into hierarchical feature representations that are not inherently interpretable. This abstraction makes it difficult to correlate these representations with human-understandable concepts [81].

Future research may explore the development of feature visualization techniques, such as activation maximization and saliency mapping, to better associate internal representations with semantic features. Integrating attention mechanisms can also offer insight into which parts of the input influence the output most. In addition, advancing concept-based interpretability methods, such as Concept Activation Vectors (CAVs), and building hybrid explanation frameworks that combine example-based reasoning with layer-level introspection could further bridge the gap between learned features and human understanding.

**6.4.1.2 Non-linear decision boundaries** Non-linear activations like ReLU enable CNNs to model complex patterns, but they create highly non-convex decision boundaries, making it difficult to trace how specific features contribute to predictions [187].

Local explanations of CNN decisions could be provided via interpretable surrogate models like decision trees or linear approximations. Quantifying the importance of each feature to the ultimate choice may also be possible with the use of techniques like Layer-wise Relevance Propagation (LRP) and SHAP values. In order to better understand model behavior, future studies should integrate counterfactual reasoning, which looks at how minor input changes could affect the output.

**6.4.1.3 High dimensionality** The extensive interconnections between neurons and layers complicate tracing the contribution of individual features to the final output, further limiting transparency [188].

Dimensionality reduction techniques (e.g., PCA, t-SNE, UMAP) and modular network designs could be explored to reduce model complexity while preserving performance. Future work may also focus on sparsity-promoting regularizations to encourage simpler and more interpretable representations. In addition, graph-based analysis of neuron interactions and network pruning methods could help isolate the most informative pathways within deep networks.

## 6.4.2 Techniques for visualizing CNN features

Visualization techniques help interpret CNNs by identifying features that influence predictions. Table 10 summarizes these methods.

## 6.4.3 Techniques for improving interpretability

Advanced methods enhance CNN interpretability by offering detailed explanations of decisions. Table 11 outlines these techniques.

**Table 10** Methods for Neural network visualization and interpretation

| Technique               | Description                                                                                                                                         | Advantages                                                       | Limitations                                                 | Applications                                |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|-------------------------------------------------------------|---------------------------------------------|
| Saliency maps           | Key regions are identified by computing output gradients with respect to input pixels, highlighting features that influence model predictions. [81] | Straightforward to compute and provides pixel-level explanations | May not provide precise explanations for deeper layers [28] | - Object detection<br>- Medical imaging     |
| Occlusion maps          | Masks parts of the input image and observes changes in prediction confidence [28]                                                                   | Offers spatial insights into feature importance                  | Computationally expensive for high-dimensional data         | - Autonomous vehicles<br>- Face recognition |
| Feature maps            | Visualizes outputs from specific CNN layers, representing extracted features [187]                                                                  | Provides insight into intermediate representations               | Limited in understanding overall model reasoning            | - Image processing<br>- Texture analysis    |
| Activation maximization | Generates inputs that maximize neuron activations [188]                                                                                             | Helps identify patterns neurons are sensitive to                 | May produce unrealistic or uninterpretable images           | Neural network debugging                    |

**Table 11** Techniques for explainable Artificial Intelligence (AI) and model interpretability

| Technique                                              | Description                                                                                                                                | Advantages                                                  | Limitations                                                    | Applications                                  |
|--------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------|----------------------------------------------------------------|-----------------------------------------------|
| Grad-CAM                                               | Class-specific heatmaps are generated by weighting gradients of target activations, highlighting key regions influencing predictions [188] | Intuitive visualizations for localized explanations         | May struggle with explaining decisions for deeper layers [189] | - Medical diagnosis<br>- Fraud detection      |
| Local Interpretable Model-agnostic Explanations (LIME) | Approximates predictions locally using an interpretable model [190]                                                                        | Flexible and model-agnostic                                 | Performance degrades with high-dimensional data                | - Healthcare<br>- Risk analysis               |
| Shapley Additive explanations (SHAP)                   | Uses game theory to assign importance scores to input features [191]                                                                       | Provides consistent and theoretically grounded explanations | Computationally intensive for large datasets                   | - Genomics<br>- Predictive analytics          |
| Integrated Gradients                                   | Computes the average gradients while varying input features from a baseline [192]                                                          | Works well with non-linear models                           | Requires selecting an appropriate baseline                     | - Text classification<br>- Image segmentation |

#### 6.4.4 Importance in healthcare and critical applications

Interpretability in sensitive domains such as healthcare is critical for ensuring trust, accountability, and regulatory compliance, enabling clinicians to validate AI-driven recommendations before making decisions:

**6.4.4.1 Building trust** Clinicians are more likely to adopt AI systems that provide clear explanations of their predictions, bridging the gap between complex models and end-users [193]. Interpretability builds a sense of confidence in the system's recommendations, making it easier for medical professionals to understand and rely on model outputs.

**6.4.4.2 Error analysis** Interpretable models help identify and correct errors, ensuring decisions are based on accurate and relevant features. This capability is crucial in high-stakes applications [194]. By tracing how a model reached a specific decision, medical practitioners can recognize if the AI was influenced by artifacts, noise, or biased patterns. This not only prevents misdiagnosis but also supports continuous model improvement and risk management.

**6.4.4.3 Regulatory compliance** Transparent AI systems align with regulatory frameworks, such as GDPR, which mandate explainability in automated decision-making [195]. Demonstrable interpretability reduces legal risk and speeds up clinical certification. Healthcare systems must often undergo audits and validations, and interpretable models facilitate this process by making AI decisions traceable and understandable to regulators.

**6.4.4.4 Improved patient outcomes** In healthcare, understanding model predictions enables clinicians to make better-informed decisions, improving diagnosis and treatment accuracy [188]. When AI systems highlight critical features contributing to a diagnosis, physicians can cross-check them with their clinical experience and existing medical records. This collaborative decision-making approach can reduce diagnostic errors and personalize treatment strategies.

## 7 Conclusion

CNNs have revolutionized AI by facilitating ground-breaking developments in fields including autonomous systems, computer vision, and natural language processing. The development of CNNs from their basic design, such as LeNet-5 and AlexNet, to cutting-edge models like ResNet and EfficientNet, which have greatly improved depth, efficiency, and accuracy, has been methodically investigated in this paper. CNNs may now perform exceptionally well on a variety of tasks thanks to developments in convolutional and pooling layers, activation functions, and regularization strategies. With noteworthy achievements in generative modeling, environmental monitoring, and medical imaging, CNNs' uses go well beyond simple image identification. These accomplishments highlight their versatility and capacity to tackle challenging issues. However, issues including data

privacy, model interpretability, and computational needs have emerged as a result of CNNs' quick development. A concentrated effort must be made to create more effective designs, incorporate explainability tools such as Grad-CAM and SHAP, and use privacy-preserving strategies in order to overcome these obstacles. In order to maximize performance and scalability, future possibilities for CNNs include investigating hybrid models, such as CNN-Transformer topologies, and integrating hardware acceleration. Furthermore, maintaining interpretability and openness is essential, particularly in high-stakes applications where responsibility and trust are crucial, such as healthcare and autonomous systems. To sum up, CNNs are well-positioned to continue leading machine learning research and applications. CNNs will remain central to AI advancements, with future research focusing on efficient architecture, hybrid models integrating Transformers, improved interpretability methods, and energy-efficient hardware acceleration. CNNs are expected to be crucial in determining the direction of intelligent systems in the future thanks to interdisciplinary cooperation and persistent research efforts.

## Appendix 1: Mathematical details of advanced pooling and activation techniques

### Advanced pooling techniques

#### Stochastic pooling

For a pooling region  $R$  with values  $\{x_1, x_2, \dots, x_n\}$ , The probability for each value:

$$P_{(x_i)} = \frac{x_i}{\sum_{j=1}^n x_j} \quad (38)$$

here  $\sum_{j=1}^n x_j$  is the sum of all values in the pooling region, ensuring that the probabilities sum to 1. One value  $x_i$  is chosen at random from region  $R$  based on the probabilities  $P_{(x_i)}$ .

#### Spectral pooling

Spectral pooling uses the Fourier domain. For computing the 2D Discrete Fourier Transform (DFT) of the input:

$$F(u, v) = \sum_{x=0}^{h-1} \sum_{y=0}^{w-1} X(x, y) e^{-j2\pi(\frac{ux}{h} + \frac{vy}{w})} \quad (39)$$

here  $h$  and  $w$  are the height and width of the input matrix,  $e^{-j2\pi(\frac{ux}{h} + \frac{vy}{w})}$  represents the complex exponential kernel that transforms spatial data into frequency components. This equation truncates high-frequency components to reduce the resolution and perform an inverse Fourier Transform to return to the spatial domain.

#### Spatial pyramid pooling (SPP)

For a feature map of size  $h \times w$  (Height and Width), divide the feature map into  $n \times n$  bins at each pyramid level  $l$ . Perform pooling each bin:

$$P_l = \text{Pooling}(\text{Region}_{(I,j)}), \quad \forall (I, j) \in l \quad (40)$$

After performing pooling operation in each bin, Spatial Pyramid Pooling concatenates pooled outputs from all levels.

## Multi-scale orderless pooling (MOP)

Computing features at different scales  $X_s$ , where  $s$  represents the scale:

$$X_s = \text{Resize}(X, s) \quad (41)$$

Resizing the image allows us to acquire features at varying granularities: finer, more localized details are captured at lower scales, while more global information is captured at larger dimensions.

Performing orderless pooling (e.g., max, average):

$$P = \text{Pooling}(X_s) \quad (42)$$

here  $P$  is the pooled feature vector at scale  $s$ . In essence, the pooling function minimizes the feature map by choosing the maximum value or averaging the values within a specified region. It can be either max-pooling or average-pooling.

## Advanced activation functions

### Exponential linear unit (ELU)

Mathematical Function for ELU:

$$F(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha(e^x - 1) & \text{if } x \leq 0 \end{cases} \quad (43)$$

where  $\alpha > 0$  is a hyperparameter (usually set to 1) that establishes the saturation point for negative inputs. ELU outputs  $x$  in the same way as ReLU for positive inputs ( $x > 0$ ). It uses an exponential function for negative inputs ( $x \leq 0$ ), smoothing the output as it gets closer to  $-\alpha$ . For negative inputs, the exponential behavior guarantees a non-zero gradient, enhancing gradient flow during training.

### Maxout

Mathematically, Maxout is defined as:

$$F(x) = \max(w_1^T x + b_1, w_2^T x + b_2, \dots, w_k^T x + b_k) \quad (44)$$

where  $w_i$  and  $b_i$  are the weights and biases for each of the  $k$  linear functions, and the function outputs the maximum value across them.

### Probout

Mathematical function for Probout:

$$F(x) = \frac{1}{1 + e^{-\lambda x}} \quad (45)$$

where  $\lambda$  is a learnable parameter that modifies the sigmoid curve's steepness. Probout adds unpredictability to the activation, especially in the negative domain, in contrast to ReLU.

## Swish

Mathematical function for Swish:

$$F(x) = x \cdot \sigma(x) \quad (46)$$

where  $\sigma(x)$  is the sigmoid function, defined as  $\sigma(x) = \frac{1}{1+e^{-x}}$ .

By multiplying the input  $x$  by the sigmoid function's output  $\sigma(x)$ , the Swish function enables the activation to have seamless transitions at both positive and negative values of  $x$ . To accommodate for non-monotonicity, the sigmoid function makes sure that the activation function returns a value between 0 and  $x$  for negative values of  $x$ . Unlike ReLU, which outputs zero for negative inputs, Swish's smooth activation reduces the likelihood of 'dead neurons' and improves learning dynamics. The gradient of the Swish function is given by:

$$F'(x) = \sigma(x) + x \cdot \sigma(x) \cdot (1 - \sigma(x)) \quad (47)$$

In contrast to sigmoid-based functions, this guarantees that the function is continuously differentiable and that gradients do not disappear as quickly.

**Acknowledgements** The authors would like to acknowledge using various AI tools, including ChatGPT, Grammarly, and Quillbot, for language refinement and editing assistance.

**Author contributions** Md. Himel Reza: Conceptualization, Methodology, Software Development, Data Curation, Formal Analysis, Writing—Original Draft; Md. Noman Biswas Sibly: Data Preprocessing, Validation, Visualization, Writing—Review and Editing; Shaikh Golam Rabbani: Investigation, Literature Review, Formal Analysis, Writing—Review and Editing; Shafayetul Huda Sadi: Resources, Experiment Design Support, Data Curation, Writing—Review and Editing; Md. Faysal Ahamed: Algorithm Optimization, Result Interpretation, Technical Support, Writing—Review and Editing; Fariya Bintay Shafi: Statistical Analysis, Figure Preparation, Proofreading; Rusab Sarmun: Supervision, Critical Review, Guidance on Experimental Framework, Project Administration; Muhammad E. H. Chowdhury: Conceptualization, Supervision, Funding Acquisition, Writing—Review and Editing, Final Approval of the Manuscript.

**Funding** Open Access funding provided by the Qatar National Library. The authors declare that no funding was obtained for this research.

**Data availability** The data and materials supporting this study's findings are available from the corresponding author upon reasonable request.

## Declarations

**Conflict of interest** The authors have no conflicts of interest to declare that are relevant to the content of this article.

**Open Access** This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

## References

- Hubel DH, Wiesel TN (1959) Receptive fields of single neurones in the cat's striate cortex. *J Physiol* 148:574–591. <https://doi.org/10.1113/JPHYSIOL.1959.SP006308>
- Fukushima K (1980) Neocognitron: a self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position. *Biol Cybern* 36:193–202. <https://doi.org/10.1007/BF00344251/METRICS>
- LeCun Y, Boser B, Denker JS, Henderson D, Howard RE, Hubbard W, Jackel LD (1989) Backpropagation applied to handwritten zip code recognition. *Neural Comput* 1:541–551. <https://doi.org/10.1162/NECO.1989.1.4.541>
- Krizhevsky A, Sutskever I, Hinton GE (2017) ImageNet classification with deep convolutional neural networks. *Commun ACM* 60:84–90. <https://doi.org/10.1145/3065386>
- He K, Zhang X, Ren S, Sun J (2016) Deep residual learning for image recognition. In: Proceedings of the IEEE computer society conference on computer vision and pattern recognition 2016-December. pp 770–778. <https://doi.org/10.1109/CVPR.2016.90>
- Tan M, Le Q V. (2019) EfficientNet: Rethinking model scaling for convolutional neural networks. In: 36th international conference on machine learning, ICML 2019 2019-June. pp 10691–10700
- Chollet F (2017) Xception: Deep learning with depthwise separable convolutions. In: Proceedings - 30th IEEE conference on computer vision and pattern recognition, CVPR 2017 2017-January. pp. 1800–1807. <https://doi.org/10.1109/CVPR.2017.195>
- Selvaraju RR, Cogswell M, Das A, Vedantam R, Parikh D, Batra D (2017) Grad-CAM: visual explanations from deep networks via gradient-based localization. In: Proceedings of the IEEE international conference on computer vision 2017-October. pp 618–626. <https://doi.org/10.1109/ICCV.2017.74>
- Chen J, Wu P, Zhang X, Xu R, Liang J (2024) Add-Vit: CNN-transformer hybrid architecture for small data paradigm processing. *Neural Process Lett*. <https://doi.org/10.1007/S11063-024-11643-8>
- Sabour S, Frosst N, Hinton GE (2017) Dynamic routing between capsules. *Adv Neural Inf Process Syst* 30:3857–3867
- LeCun Y, Bottou L, Bengio Y, Haffner P (1998) Gradient-based learning applied to document recognition. *Proc IEEE* 86:2278–2323. <https://doi.org/10.1109/5.726791>
- Lee D-H (2013) Pseudo-label: the simple and efficient semi-supervised learning method for deep neural networks
- Yosinski J, Clune J, Bengio Y, Lipson H (2014) How transferable are features in deep neural networks? *Adv Neural Inf Process Syst* 4:3320–3328
- Tahmid M, Alam MS, Rao N, Ashrafi KMA (2016) Image-to-image translation with conditional adversarial networks. In: Proceedings of 2023 IEEE 9th international women in engineering (WIE) conference on electrical and computer engineering, WIECON-ECE 2023. pp 468–472. <https://doi.org/10.1109/WIECON-ECE60392.2023.10456447>
- Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, Kaiser Ł, Polosukhin I (2017) Attention is all you need. *Adv Neural Inf Process Syst* 30:5999–6009
- Huang G, Liu Z, Van Der Maaten L, Weinberger KQ (2016) Densely connected convolutional networks. In: Proceedings - 30th IEEE conference on computer vision and pattern recognition, CVPR 2017 2017. pp 2261–2269. <https://doi.org/10.1109/CVPR.2017.243>
- Redmon J, Farhadi A (2018) YOLOv3: an incremental improvement. <https://doi.org/10.48550/arXiv.1804.02767>
- Madry A, Makelov A, Schmidt L, Tsipras D, Vladu A (2017) Towards deep learning models resistant to adversarial attacks. In: 6th international conference on learning representations, ICLR 2018 - conference track proceedings. <https://doi.org/10.48550/arXiv.1706.06083>
- Tan M, Pang R, Le Q V. (2019) EfficientDet: scalable and efficient object detection. In: Proceedings of the IEEE computer society conference on computer vision and pattern recognition. pp 10778–10787. <https://doi.org/10.1109/CVPR42600.2020.01079>
- Dosovitskiy A, Beyer L, Kolesnikov A, Weissenborn D, Zhai X, Unterthiner T, Dehghani M, Minderer M, Heigold G, Gelly S, Uszkoreit J, Houlsby N (2020) An image is worth 16×16 words: transformers for image recognition at scale. In: ICLR 2021 - 9th international conference on learning representations
- Liu Z, Lin Y, Cao Y, Hu H, Wei Y, Zhang Z, Lin S, Guo B (2021) Swin transformer: Hierarchical vision transformer using shifted windows. In: Proceedings of the IEEE international conference on computer vision. pp 9992–10002. <https://doi.org/10.1109/ICCV48922.2021.00986>
- Chatterjee S, Tummala P, Speck O, Nürnberger A (2023) Complex network for complex problems: a comparative study of CNN and complex-valued CNN. In: 5th IEEE international image processing, applications and systems conference, IPAS 2022. <https://doi.org/10.1109/IPAS55744.2022.10053060>
- Omar M (2022) New approach to malware detection using optimized convolutional neural network. Springer, New York, pp 13–35. [https://doi.org/10.1007/978-3-031-15893-3\\_2](https://doi.org/10.1007/978-3-031-15893-3_2)
- Bougourzi F, Dornaika F, Nakib A, Distant C, Taleb-Ahmed A (2023) 2D and 3D CNN-based fusion approach for COVID-19 severity prediction from 3D CT-scans

25. Yunusa í mí H, Qin í mí S, Hamman Adama Chukkol í mí A, Abdu Yusuf í mí A, Bello I, mí í, Lawan í mí A (2024) Exploring the synergies of hybrid CNNs and ViTs architectures for computer vision: a survey
26. Howard AG, Zhu M, Chen B, Kalenichenko D, Wang W, Weyand T, Andreetto M, Adam H (2017) MobileNets: efficient convolutional neural networks for mobile vision applications
27. Yamaguchi K, Sakamoto K, Akabane T, Fujimoto Y (1990) A neural network for speaker-independent isolated word recognition. In: 1st international conference on spoken language processing, ICSLP 1990. pp 1077–1080. <https://doi.org/10.21437/ICSLP.1990-282>
28. Zeiler MD, Fergus R (2014) Visualizing and understanding convolutional networks. Comput Vision–ECCV 2014 8689:818–833. [https://doi.org/10.1007/978-3-319-10590-1\\_53](https://doi.org/10.1007/978-3-319-10590-1_53)
29. Christlein V, Spranger L, Seuret M, Nicolaou A, Kral P, Maier A (2019) Deep generalized max pooling. In: Proceedings of the international conference on document analysis and recognition, ICDAR. pp 1090–1096. <https://doi.org/10.1109/ICDAR.2019.00177>
30. Lin M, Chen Q, Yan S (2013) Network in network
31. Yu D, Wang H, Chen P, Wei Z (2014) Mixed pooling for convolutional neural networks. Lect Notes Comput Sci (Including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics) 8818:364–375. [https://doi.org/10.1007/978-3-319-11740-9\\_34](https://doi.org/10.1007/978-3-319-11740-9_34)
32. Zeiler MD, Fergus R (2013) Stochastic pooling for regularization of deep convolutional neural networks. In: 1st international conference on learning representations, ICLR 2013 - conference track proceedings
33. Defferrard M, Bresson X, Vandergheynst P (2016) Convolutional neural networks on graphs with fast localized spectral filtering. Adv Neural Inf Process Syst 29:3844–3852
34. Rippel O, Snoek J, Adams RP (2015) Spectral representations for convolutional neural networks. Adv Neural Inf Process Syst 28:2449–2457
35. He K, Zhang X, Ren S, Sun J (2014) Spatial pyramid pooling in deep convolutional networks for visual recognition. In: Lecture notes in computer science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics) 8691 LNCS. pp 346–361. [https://doi.org/10.1007/978-3-319-10578-9\\_23](https://doi.org/10.1007/978-3-319-10578-9_23)
36. Ren S, He K, Girshick R, Sun J (2015) Faster R-CNN: towards real-time object detection with region proposal networks. IEEE Trans Pattern Anal Mach Intell 39:1137–1149. <https://doi.org/10.1109/TPAMI.2016.2577031>
37. Gong Y, Wang L, Guo R, Lazebnik S (2014) Multi-scale orderless pooling of deep convolutional activation features. In: Lecture notes in computer science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics) 8695 LNCS: pp. 392–407. [https://doi.org/10.1007/978-3-319-10584-0\\_26](https://doi.org/10.1007/978-3-319-10584-0_26)
38. Dubey S, Singh S, Neurocomputing BC (2022) Activation functions in deep learning: a comprehensive survey and benchmark. Elsevier, Amsterdam
39. Glorot X, Bordes A, Bengio Y (2011) Deep sparse rectifier neural networks. In: International conference on artificial intelligence and statistics
40. Rumelhart DE, Hinton GE, Williams RJ (1986) Learning representations by back-propagating errors. Nature 323(6088):533–536. <https://doi.org/10.1038/323533a0>
41. Zhang X, Zhang Z, Bae H (2019) Compact cluster-based balanced distribution adaptation for transfer learning. In: Proceedings of the international joint conference on neural networks 2019-July. <https://doi.org/10.1109/IJCNN.2019.8852238>
42. Nair V, Hinton GE (2010) Rectified linear units improve restricted Boltzmann machines. In: International conference on machine learning
43. Maas A, Hannun A, icml AN-Proc, 2013 undefined (2013) Rectifier nonlinearities improve neural network acoustic models. awnihannun.comAL Maas, AY Hannun, AY NgProc icml, 2013 awnihannun.com
44. Xu B, Wang N, Kong H, Chen T, Li M (2015) Empirical evaluation of rectified activations in convolutional network. <https://doi.org/10.48550/arXiv.1505.00853>
45. He K, Zhang X, Ren S, IEEE JS-P (2015) Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. openaccess.thecvf.comK He, X Zhang, S Ren, J Sun. In: Proceedings of the IEEE international conference on computer, 2015•openaccess.thecvf.com
46. Jiang T, Cheng J (2019) Target recognition based on CNN with LeakyReLU and PReLU activation functions. In: Proceedings - 2019 international conference on sensing, diagnostics, prognostics, and control, SDPC 2019. pp 718–722. <https://doi.org/10.1109/SDPC.2019.00136>
47. Clevert DA, Unterthiner T, Hochreiter S (2015) Fast and accurate deep network learning by exponential linear units (ELUs). In: 4th international conference on learning representations, ICLR 2016 - conference track proceedings
48. Grelsson B, Felsberg M (2018) Improved learning in convolutional neural networks with shifted exponential linear units (ShELUs). In: Proceedings - international conference on pattern recognition. pp 517–522. <https://doi.org/10.1109/ICPR.2018.8545104>
49. Goodfellow I, Warde-Farley D, Mirza M, Courville AC, Bengio Y (2013) Maxout networks. In: International conference on machine learning

50. Springenberg JT, Riedmiller M (2013) Improving deep neural networks with probabilistic maxout units. In: 2nd international conference on learning representations, ICLR 2014 - workshop track proceedings
51. Shridhar K, Lee J, Hayashi H, Mehta P, Iwana BK, Kang S, Uchida S, Ahmed S, Dengel A (2019) ProbAct: a probabilistic activation function for deep neural networks
52. Ramachandran P, Zoph B, Le Google Brain Q V (2017) Searching for activation functions. In: 6th international conference on learning representations, ICLR 2018 - workshop track proceedings
53. Zhang Z, Liu J, Liu G, Wang J, Zhang J (2023) Robustness verification of Swish neural networks embedded in autonomous driving systems. *IEEE Trans Comput Soc Syst* 10:2041–2050. <https://doi.org/10.1109/TCSS.2022.3179659>
54. Huang YH, Proesmans M, Gool L Van (2023) Padding investigations for CNNs in scene parsing tasks. In: Proceedings of MVA 2023 - 18th international conference on machine vision and applications. <https://doi.org/10.23919/MVA57639.2023.10216084>
55. Nguyen AD, Choi S, Kim W, Ahn S, Kim J, Lee S (2019) Distribution padding in convolutional neural networks. In: Proceedings - international conference on image processing, ICIP. pp 4275–4279. <https://doi.org/10.1109/ICIP.2019.8803537>
56. Montúfar G, Pascanu R, Cho K, and Bengio Y (2014) On the number of linear regions of deep neural networks. *Adv Neural Inf Process Syst* 27
57. He K and Sun J (2015) Convolutional neural networks at constrained time cost. In: Proceedings of the IEEE conference on computer vision and pattern recognition
58. Dumoulin V and Visin F (2016) A guide to convolution arithmetic for deep learning. arXiv preprint [arXiv:1603.07285](https://arxiv.org/abs/1603.07285)
59. Sainath T, Interspeech CP (2015) Convolutional neural networks for small-footprint keyword spotting. <https://doi.org/10.21437/Interspeech.2015-352>
60. LeCun Y, Bengio Y (2015) Deep learning. *Nature* 521:436–444
61. Goodfellow I, Bengio Y, Courville A, Heaton J (2017) Ian Goodfellow, Yoshua Bengio, and Aaron Courville: Deep learning. *Genet Program Evol Mach* 19:305–307. <https://doi.org/10.1007/S10710-017-9314-Z>
62. Montavon G, Braun M (2011) Kernel analysis of deep networks. *J Mach Learn Res* 12:2563–2581
63. Montavon G, Braun ML (2011) Kernel analysis of deep networks klaus-robert Muller \*
64. Narkhede M, Mahajan S, Bartakke P, Sutaone M (2024) Towards compressed and efficient CNN architectures via pruning. *Discover Comput*. <https://doi.org/10.1007/S10791-024-09463-4>
65. Cortes C, Vapnik V (1995) Support-vector networks. *Mach Learn* 20:273–297. <https://doi.org/10.1007/BF00994018>
66. Huang GB, Mattar MA, Berg TL, Learned-Miller E (2008) Labeled faces in the wild: a database for studying face recognition in unconstrained environments
67. Srivastava N, Hinton G, Krizhevsky A, Sutskever I, Salakhutdinov R (2014) Dropout: a simple way to prevent neural networks from overfitting. *J Mach Learn Res* 15(1):1929–1958
68. Hadsell R, Chopra S, LeCun Y (2006) Dimensionality reduction by learning an invariant mapping. *Proc IEEE Comput Soc Conf Comput Vision Pattern Recogn* 2:1735–1742. <https://doi.org/10.1109/CVPR.2006.100>
69. Bromley J, Bentz JW, Bottou L, Guyon I, Lecun Y, Moore C, Säckinger E, Shah R (1993) Signature verification using a “SIAMESE” time delay neural network. *Int J Pattern Recognit Artif Intell* 07:669–688. <https://doi.org/10.1142/S0218001493000339>
70. Hermans A, Beyer L, Leibe B (2017) Defense of the triplet loss for person re-identification
71. Parkhi O, Vedaldi A, Zisserman A (2015) Deep face recognition. In: BMVC 2015-proceedings of the british machine vision conference 2015. British Machine Vision Association
72. Kingma DP, Welling M (2014) Auto-encoding variational bayes. In: 2nd international conference on learning representations, ICLR 2014 - conference track proceedings. <https://doi.org/10.61603/ceas.v2i1.33>
73. Rezende DJ, Mohamed S, Wierstra D (2014) Stochastic backpropagation and approximate inference in deep generative models. In: 31st international conference on machine learning, vol 4. ICML, pp 3057–3070
74. Bhakta S, Nandi U, Mahapatra KR, Singh MM, Noorwali A (2024) SWOSBC: a novel optimizer for learning convolutional neural network. *IEEE Access*. <https://doi.org/10.1109/ACCESS.2024.3481640>
75. Ruder S (2016) An overview of gradient descent optimization algorithms
76. Bottou L (2010) Large-scale machine learning with stochastic gradient descent. In: Proceedings of COMPSTAT 2010 - 19th international conference on computational statistics, Keynote, invited and contributed papers. pp 177–186. [https://doi.org/10.1007/978-3-7908-2604-3\\_16](https://doi.org/10.1007/978-3-7908-2604-3_16)
77. Qian N (1999) On the momentum term in gradient descent learning algorithms. *Neural Netw* 12(1):145–151
78. Duchi J, Hazan E, Singer Y (2011) Adaptive subgradient methods for online learning and stochastic optimization. *J Mach Learn Res* 12:2121–2159
79. Dogo EM, Afolabi OJ, Nwulu NI, Twala B, Aigbavboa CO (2018) A comparative analysis of gradient descent-based optimization algorithms on convolutional neural networks. *Proc Int Conf Comput Tech Electron Mech Syst CTEMS* 2018:92–99. <https://doi.org/10.1109/CTEMS.2018.8769211>

80. Zeiler MD (2012) ADADELTA: an adaptive learning rate method
81. Simonyan K, Zisserman A (2014) Very deep convolutional networks for large-scale image recognition. In: 3rd international conference on learning representations. ICLR 2015 - conference track proceedings
82. Szegedy C, Liu W, Jia Y, Sermanet P, Reed S, Anguelov D, Erhan D, Vanhoucke V, Rabinovich A (2015) Going deeper with convolutions. In: Proceedings of the IEEE computer society conference on computer vision and pattern recognition. pp 1–9. <https://doi.org/10.1109/CVPR.2015.7298594>
83. Xie S, Girshick R, Dollár P, Tu Z, He K (2017) Aggregated residual transformations for deep neural networks. In: Proceedings - 30th IEEE conference on computer vision and pattern recognition, CVPR. pp 5987–5995. <https://doi.org/10.1109/CVPR.2017.634>
84. Zhang Y, Tian Y, Kong Y, Zhong B, Fu Y (2018) Residual dense network for image super-resolution
85. Zoph B, Brain G, Vasudevan V, Shlens J, Le Google Brain QV (2018) Learning transferable architectures for scalable image recognition. In: Proceedings of the IEEE conference on computer vision and pattern recognition
86. Yu F, Koltun V (2016) Multi-scale context aggregation by dilated convolutions. In: 4th international conference on learning representations, ICLR 2016 - conference track proceedings
87. Gu J, Wang Z, Kuen J, Ma L, Shahroudy A, Shuai B, Chen T et al (2018) Recent advances in convolutional neural networks. *Pattern Recogn* 77:354–377
88. Szegedy C, Ioffe S, Vanhoucke V (2017) Inception-v4, inception-resnet and the impact of residual connections on learning. In: Proceedings of the AAAI conference on artificial intelligence
89. Szegedy C, Vanhoucke V, Ioffe S, Shlens J, Wojna Z (2015) Rethinking the inception architecture for computer vision. In: Proceedings of the IEEE computer society conference on computer vision and pattern recognition. pp 2818–2826. <https://doi.org/10.1109/CVPR.2016.308>
90. Visin F, Romero A, Ciccone M, Kastner K, Cho K, Matteucci M, Bengio Y, Courville A Reseg: a recurrent neural network-based model for semantic segmentation. [cv-foundation.org](http://cv-foundation.org)
91. Lin M, Chen Q, Yan S (2013) Network in network. In: 2nd international conference on learning representations, ICLR 2014 - conference track proceedings. <https://doi.org/10.48550/arXiv.1312.4400>
92. Gholamalinezhad H, Khosravi H (2020) Pooling methods in deep neural networks, a review
93. Sermanet P, Chintala S (2012) Convolutional neural networks applied to house numbers digit classification. In: Proceedings of the 21st international conference on pattern recognition (ICPR2012)
94. Kwak S, Hong S (2017) Weakly supervised semantic segmentation using superpixel pooling network. In: Proceedings of the AAAI conference on artificial intelligence
95. Rasamoelina AD, Adjailia F, Sincak P (2020) A review of activation function for artificial neural network. In: SAMI 2020 - IEEE 18th world symposium on applied machine intelligence and informatics, proceedings. pp 281–286. <https://doi.org/10.1109/SAMI48414.2020.9108717>
96. Dubey SR, Singh SK, Chaudhuri BB (2022) Activation functions in deep learning: a comprehensive survey and benchmark. *Neurocomputing* 503:92–108. <https://doi.org/10.1016/J.NEUCOM.2022.06.111>
97. Li Z, Liu F, Yang W, Peng S, Zhou J (2021) A survey of convolutional neural networks: analysis, applications, and prospects. *IEEE Trans Neural Netw Learn Syst* 33(12):6999–7019
98. Willmott CJ, Matsuura K (2005) Advantages of the mean absolute error (MAE) over the root mean square error (RMSE) in assessing average model performance. *Clim Res* 30:79–82. <https://doi.org/10.3354/CR030079>
99. Hastie T, Tibshirani R, Friedman J (2009) The Elements of Statistical. Learning. <https://doi.org/10.1007/978-0-387-84858-7>
100. (2007) Pattern recognition and machine learning. *J Electron Imaging* 16:049901. <https://doi.org/10.1117/1.2819119>
101. Guo C, Pleiss G, Sun Y, Weinberger KQ (2017) On calibration of modern neural networks. In: 34th international conference on machine learning, vol 3. ICML, pp 2130–2143
102. Deng J, Guo J, Yang J, Xue N, Kotsia I, Zafeiriou S (2022) ArcFace: additive angular margin loss for deep face recognition. *IEEE Trans Pattern Anal Mach Intell* 44:5962–5979. <https://doi.org/10.1109/TPAMI.2021.3087709>
103. Chen W, Chen X, Zhang J, Huang K (2017) A multi-task deep network for person re-identification. *Proc AAAI Conf Artif Intell* 31:3988–3994. <https://doi.org/10.1609/AAAI.V31I1.11201>
104. Xie X (2024) A hybrid CNN-LSTM architecture for enhanced music genre classification. In: 2024 5th international conference on machine learning and computer application (ICMLCA)
105. Demiss BA, Elsaigh WA (2024) Application of novel hybrid deep learning architectures combining convolutional neural networks (CNN) and recurrent neural networks (RNN): construction duration estimates prediction considering preconstruction uncertainties. *Eng Res Express* 6(3):032102. <https://doi.org/10.1088/2631-8695/ad6ca7>
106. Nandhini I, Prasanth LL, Nagalakshmi T, and Manjula D (2024) Integrating convolutional and recurrent networks for image caption generation: a unified approach. In: 2024 15th international conference on computing communication and networking technologies (ICCCNT). pp 1–5. IEEE
107. Sitanggang APP (2024) Hybrid CNN and RNNs model for sentiment analysis. *Proxies J Inf* 8(1):1–12

108. Cao Z, Tang S (2024) IRNet: an information retrieval model based on dual-stream fusion of CNN and transformer. In: 2024 5th international conference on machine learning and computer application
109. Wu L, Zhang M, Piao Y, Yao Z, Sun W, Tian F, Lu H (2024) CNN-transformer rectified collaborative learning for medical image segmentation. *IEEE Trans Circ Syst Video Technol* 35:4072–4086
110. Pereira GA, Hussain M (2024) A review of transformer-based models for computer vision tasks: capturing global context and spatial relationships
111. Jung M, Lee J, Intelligence JK-A (2024) A lightweight CNN-transformer model for learning traveling salesman problems. *Appl Intell* 54:7982–7993. <https://doi.org/10.1007/s10489-024-05603-x>
112. Vats S, Singh CR, and Mehta S (2024) Next-generation diagnostic tools: the role of hybrid CNN-transformer models in liver cancer detection. In: 2024 15th international conference on computing communication and networking technologies (ICCCNT). IEEE, pp 1–5
113. Mehta S, Rastegari M (2021) MobileViT: light-weight, general-purpose, and mobile-friendly vision transformer. In: ICLR 2022 - 10th international conference on learning representations
114. Bronstein MM, Bruna J, LeCun Y, Szlam A, Vandergheynst P (2016) Geometric deep learning: going beyond Euclidean data. *IEEE Signal Process Mag*. <https://doi.org/10.1109/MSP.2017.2693418>
115. Battaglia PW, Hamrick JB, Bapst V, Sanchez-Gonzalez A, Zambaldi V, Malinowski M, Tacchetti A, Raposo D, Santoro A, Faulkner R, Gulcehre C, Song F, Ballard A, Gilmer J, Dahl G, Vaswani A, Allen K, Nash C, Langston V, Dyer C, Heess N, Wierstra D, Kohli P, Botvinick M, Vinyals O, Li Y, Pascanu R (2018) Relational inductive biases, deep learning, and graph networks
116. Kipf TN, Welling M (2016) Semi-supervised classification with graph convolutional networks
117. Wu Z, Pan S, Chen F, Long G, Zhang C, Yu PS (2019) A comprehensive survey on graph neural networks. *IEEE Trans Neural Netw Learn Syst*. <https://doi.org/10.1109/TNNLS.2020.2978386>
118. Parisot S, Ktena SI, Ferrante E, Lee M, Guerrero R, Glocker B, Rueckert D (2018) Disease prediction using graph convolutional networks: application to Autism Spectrum Disorder and Alzheimer's disease. *Med Image Anal* 48:117–130. <https://doi.org/10.1016/j.media.2018.06.001>
119. Qi CR, Su H, Mo K, Guibas LJ (2017) PointNet: deep learning on point sets for 3D classification and segmentation
120. Wang Y, Sun Y, Liu Z, Sarma SE, Bronstein MM, Solomon JM (2019) Dynamic graph CNN for learning on point clouds. *ACM Trans Graph*. <https://doi.org/10.1145/3326362>
121. Zhang Y, Rabbat M (2018) A graph-CNN for 3D point cloud classification
122. Hu Q, Yang B, Xie L, Rosa S, Guo Y, Wang Z, Trigoni N, Markham A (2019) RandLA-Net: efficient semantic segmentation of large-scale point clouds
123. Hamilton WL, Ying R, Leskovec J (2017) Inductive representation learning on large graphs
124. Fey M, Lenssen JE (2019) Fast graph representation learning with PyTorch geometric
125. Weng W, Zhu X (2015) U-Net: convolutional networks for biomedical image segmentation. *IEEE Access* 9:16591–16603. <https://doi.org/10.1109/ACCESS.2021.3053408>
126. Setio AAA, Ciompi F, Litjens G, Gerke P, Jacobs C, Van Riel SJ, Wille MMW, Naqibullah M, Sanchez CI, Van Ginneken B (2016) Pulmonary nodule detection in CT images: false positive reduction using multi-view convolutional networks. *IEEE Trans Med Imaging* 35:1160–1169. <https://doi.org/10.1109/TMI.2016.2536809>
127. Long J, Shelhamer E, and Darrell T (2015) Fully convolutional networks for semantic segmentation
128. Geiger A, Lenz P, Urtasun R (2012) Are we ready for autonomous driving? The KITTI vision benchmark suite. In: Proceedings of the IEEE computer society conference on computer vision and pattern recognition, pp 3354–3361. <https://doi.org/10.1109/CVPR.2012.6248074>
129. Islam MR, Ahamed MF, Islam MR, Nahiduzzaman M, Ahsan M (2025) Detection, localization, segmentation, and classification in colorectal cancer screening using deep learning: a systematic review. *Biomed Signal Process Control* 110:108202. <https://doi.org/10.1016/J.BSPC.2025.108202>
130. Ahamed MF, Nahiduzzaman M, Islam MR, Naznine M, Arselene Ayari M, Khandakar A, Haider J (2024) Detection of various gastrointestinal tract diseases through a deep learning method with ensemble ELM and explainable AI. *Expert Syst Appl* 256:124908. <https://doi.org/10.1016/J.ESWA.2024.124908>
131. Ahamed MF, Shafi FB, Nahiduzzaman M, Ayari MA, Khandakar A (2025) Interpretable deep learning architecture for gastrointestinal disease detection: a Tri-stage approach with PCA and XAI. *Comput Biol Med* 185:109503. <https://doi.org/10.1016/J.COMPBIOMED.2024.109503>
132. Ahamed MF, Nahiduzzaman M, Mahmud G, Shafi FB, Ayari MA, Khandakar A, Abdullah-Al-Wadud M, Islam SMR (2025) Improving Malaria diagnosis through interpretable customized CNNs architectures. *Sci Rep*. <https://doi.org/10.1038/s41598-025-90851-1>
133. Sarkar O, Islam MR, Syfullah MK, Islam MT, Ahamed MF, Ahsan M, Haider J (2023) Multi-scale CNN: an explainable AI-integrated unique deep learning framework for lung-affected disease classification. *Technologies*. <https://doi.org/10.3390/technologies11050134>
134. Badawi MB, Salah B, Fawaz W (2025) Deep learning-based pneumonia detection using chest x-ray images. *Int J Telecommun* 05:1–16. <https://doi.org/10.21608/IJT.2025.346338.1071>

135. Badawi M, Elgohary R, Tarek M, EzzAlRegal M, Ahmed A, Samir A, Ehab N (2024) Skin cancer classification and segmentation using deep learning. *Int J Telecommun* 04:1–23. <https://doi.org/10.21608/IJT.2024.280957.1045>
136. Ahamed MF et al (2024) Interpretable deep learning model for tuberculosis detection using X-ray images. *Surveillance Prevent Control Infect Dis*. [https://doi.org/10.1007/978-3-031-59967-5\\_8](https://doi.org/10.1007/978-3-031-59967-5_8)
137. Hossain MdM, Islam MdR, Ahamed MdF, Ahsan M, Haider J (2024) A collaborative federated learning framework for lung and colon cancer classifications. *Technologies* 12:151. <https://doi.org/10.3390/TECHNOLOGIES12090151>
138. Redmon J, Divvala S, Girshick R, Farhadi A (2016) You only look once: unified, real-time object detection. 779–788
139. Badrinarayanan V, Kendall A, Cipolla R (2017) Segnet: a deep convolutional encoder-decoder architecture for image segmentation. *IEEE Trans Pattern Anal Mach Intell* 39:2481–2495. <https://doi.org/10.1109/TPAMI.2016.2644615>
140. Ku J, Mozifian M, Lee J, Harakeh A, and Waslander SL (2018) Joint 3d proposal generation and object detection from view aggregation. In: 2018 IEEE/RSJ international conference on intelligent robots and systems (IROS). IEEE, pp 1–8
141. Kim Y (2014) Convolutional neural networks for sentence classification. In: EMNLP 2014 - 2014 conference on empirical methods in natural language processing, proceedings of the conference. pp 1746–1751. <https://doi.org/10.3115/v1/d14-1181>
142. Dos Santos C and Gatti M (2014) Deep convolutional neural networks for sentiment analysis of short texts. In: Proceedings of COLING 2014, the 25th international conference on computational linguistics: technical papers. pp 69–78
143. Gehring J, Auli M, Grangier D, Yarats D, Dauphin YN (2017) Convolutional sequence to sequence learning. In: International conference on machine learning, pp 1243–1252
144. Tran D, Bourdev L, Fergus R, Torresani L, and Paluri M (2015) Learning spatiotemporal features with 3D convolutional networks. In: Proceedings of the IEEE international conference on computer vision. pp 4489–4497
145. Carreira J and Zisserman A (2017) Quo vadis, action recognition? A new model and the kinetics dataset. In: Proceedings of the IEEE conference on computer vision and pattern recognition, pp 6299–6308
146. Feichtenhofer C, Pinz A, and Zisserman A (2016) Convolutional two-stream network fusion for video action recognition. In: Proceedings of the IEEE conference on computer vision and pattern recognition. pp 1933–1941
147. Sultani W, Chen C, and Shah M (2018) Real-world anomaly detection in surveillance videos. In: Proceedings of the IEEE conference on computer vision and pattern recognition. pp 6479–6488
148. Goodfellow IJ, Pouget-Abadie J, Mirza M, Xu B, Warde-Farley D, Ozair S, Courville A, Bengio Y (2014) Generative adversarial nets. *Adv Neural Inf Process Syst* 27
149. Karras T, Aila T, Laine S, and Lehtinen J (2017) Progressive growing of gans for improved quality, stability, and variation. [arXiv:1710.10196](https://arxiv.org/abs/1710.10196)
150. Arğun H (2023) Variational autoencoder-based anomaly detection in time series data for inventory record inaccuracy. *Turk J Electr Eng Comput Sci*. <https://doi.org/10.55730/1300-0632.3977>
151. Ledig C, Theis L, Huszár F, Caballero J, Cunningham A, Acosta A, Shi W et al (2017) Photo-realistic single image super-resolution using a generative adversarial network. In: Proceedings of the IEEE conference on computer vision and pattern recognition. pp 4681–4690
152. Michel JO (2019) Deep learning on sustainable development. *Encycl Sustain Higher Educ*. [https://doi.org/10.1007/978-3-030-11352-0\\_207](https://doi.org/10.1007/978-3-030-11352-0_207)
153. Kamilaris A, Prenafeta-Boldú FX (2018) Deep learning in agriculture: a survey. *Comput Electron Agric* 147:70–90
154. Emanuel K (2005) Increasing destructiveness of tropical cyclones over the past 30 years. *Nature* 436(7051):686–688. <https://doi.org/10.1038/nature03906>
155. Ahamed MdF, Salam A, Nahiduzzaman Md, Abdullah-Al-Wadud M, Islam SMR (2024) Streamlining plant disease diagnosis with convolutional neural networks and edge devices. *Neural Comput Appl* 36:18445–18477. <https://doi.org/10.1007/s00521-024-10152-y>
156. Nahiduzzaman M, Ahamed MF, Naznine M, Karim MJ, Kibria HB, Ayari MA, Khandakar A, Ashraf A, Ahsan M, Haider J (2025) An automated waste classification system using deep learning techniques: toward efficient waste recycling and environmental sustainability. *Knowl Based Syst* 310:113028. <https://doi.org/10.1016/J.KNOSYS.2025.113028>
157. Norouzzadeh M, Norouzzadeh MS, Nguyen A, Kosmala M, Swanson A, Palmer MS, Packer C, Clune J (2018) Automatically identifying, counting, and describing wild animals in camera-trap images with deep learning. *Proc Natl Acad Sci U S A* 115:E5716–E5725. <https://doi.org/10.1073/pnas.1719367115>
158. Bell S, Bala K (2015) Learning visual similarity for product design with convolutional neural networks. *ACM Trans Graphics (TOG)* 34(4):1–10
159. Han X, Wu Z, Wu Z, Yu R, and Davis LS (2018) Viton: an image-based virtual try-on network. In: Proceedings of the IEEE conference on computer vision and pattern recognition. pp 7543–7552

160. Covington P, Adams J, and Sargin E (2016) Deep neural networks for youtube recommendations. In: Proceedings of the 10th ACM conference on recommender systems. pp 191–198
161. Hidasi B, Karatzoglou A, Baltrunas L, Tikk D (2016) Session-based recommendations with recurrent neural networks. In: 4th international conference on learning representations, ICLR 2016 - conference track proceedings
162. Russakovsky O, Deng J, Su H, Krause J, Satheesh S, Ma S, Huang Z, Karpathy A, Khosla A, Bernstein M, Berg AC, Fei-Fei L, Russakovsky O, Deng J, Su H, Krause J, Satheesh S, Ma S, Huang Z, Karpathy A, Khosla A, Bernstein M, Berg AC, Fei-Fei L (2014) ImageNet large scale visual recognition challenge. *Int J Comput Vis* 115:211–252. <https://doi.org/10.1007/s11263-015-0816-y>
163. Wang Z, Wang X, Wang G (2018) Learning fine-grained features via a CNN tree for large-scale classification. *Neurocomputing* 275:1231–1240
164. Minervini M, Fischbach A, Scharr H, Tsaftaris SA (2016) Finely-grained annotated datasets for image-based plant phenotyping. *Pattern Recogn Lett* 81:80–89
165. Branson S, Van Horn G, Belongie S, Perona P (2014) Bird species categorization using pose normalized deep convolutional nets. In: BMVC 2014 - proceedings of the British machine vision conference 2014
166. Zhou Q, Zheng B, Zhu W, Latecki LJ (2016) Multi-scale context for scene labeling via flexible segmentation graph. *Pattern Recogn* 59:312–324
167. Shuai B, Wang G, Zuo Z, Wang B, Zhao L (2015) Integrating parametric and non-parametric models for scene labeling. In: Proceedings of the IEEE conference on computer vision and pattern recognition. pp 4249–4258. <https://doi.org/10.1109/CVPR.2015.7299053>
168. Shuai B, Zuo Z, Wang G (2015) Quaddirectional 2D-recurrent neural networks for image labeling. *IEEE Signal Process Lett* 22:1990–1994. <https://doi.org/10.1109/LSP.2015.2441781>
169. Mostajabi M, Yadollahpour P, Shakhnarovich G (2014) Feedforward semantic segmentation with zoom-out features. In: Proceedings of the IEEE computer society conference on computer vision and pattern recognition. pp 3376–3385. <https://doi.org/10.1109/CVPR.2015.7298959>
170. Chen L-C, Papandreou G, Kokkinos I, Murphy K, Yuille AL (2014) Semantic image segmentation with deep convolutional nets and fully connected CRFs. 1–12
171. Li Y, Wang S, Tian Q, Ding X (2015) Feature representation for statistical-learning-based object detection: a review. *Pattern Recognit* 48:3542–3559. <https://doi.org/10.1016/J.PATCOG.2015.04.018>
172. Girshick R, Iandola F, Darrell T, Malik J (2014) Deformable part models are convolutional neural networks. In: Proceedings of the IEEE computer society conference on computer vision and pattern recognition. pp 437–446. <https://doi.org/10.1109/CVPR.2015.7298641>
173. Uijlings JRR, Van De Sande KEA, Gevers T, Smeulders AWM (2013) Selective search for object recognition. *Int J Comput Vis* 104:154–171. <https://doi.org/10.1007/S11263-013-0620-5>
174. Zhang C, Yao C, Shi B, Bai X (2015) Automatic discrimination of text and non-text natural images. In: Proceedings of the international conference on document analysis and recognition, ICDAR. pp 886–890. <https://doi.org/10.1109/ICDAR.2015.7333889>
175. Goodfellow IJ, Bulatov Y, Ibarz J, Arnoud S, Shet V (2013) Multi-digit number recognition from street view imagery using deep convolutional neural networks. In: 2nd international conference on learning representations, ICLR 2014 - conference track proceedings
176. Jaderberg M, Simonyan K, Vedaldi A, Zisserman A (2014) Deep structured output learning for unconstrained text recognition. In: 3rd international conference on learning representations, ICLR 2015 - conference track proceedings
177. Jaderberg M, Vedaldi A, Zisserman A (2014) Deep features for text spotting. *Lect Notes Comput Sci (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)* 8692:512–528. [https://doi.org/10.1007/978-3-319-10593-2\\_34](https://doi.org/10.1007/978-3-319-10593-2_34)
178. Tousch AM, Herbin S, Audibert JY (2012) Semantic hierarchies for image annotation: a survey. *Pattern Recognit* 45:333–345. <https://doi.org/10.1016/J.PATCOG.2011.05.017>
179. Wang Z, Wang X, Wang G (2018) Learning fine-grained features via a CNN tree for Large-scale classification. *Neurocomputing* 275:1231–1240. <https://doi.org/10.1016/J.NEUCOM.2017.09.061>
180. Yan Z, Jagadeesh V, Decoste D, Di W, Piramuthu R (2015) HD-CNN: hierarchical deep convolutional neural network for image classification
181. Berg T, Liu J, Woo Lee S, Alexander ML, Jacobs DW, Belhumeur PN (2014) Birdsnap: large-scale fine-grained visual categorization of birds. 2011–2018
182. Yang L, Luo P, Loy CC, Tang X (2015) A large-scale car dataset for fine-grained categorization and verification. In: Proceedings of the IEEE computer society conference on computer vision and pattern recognition. pp 3973–3981. <https://doi.org/10.1109/CVPR.2015.7299023>
183. Xie GS, Zhang XY, Yang W, Xu M, Yan S, Liu CL (2017) LG-CNN: from local parts to global discrimination for fine-grained recognition. *Pattern Recognit* 71:118–131. <https://doi.org/10.1016/J.PATCOG.2017.06.002>

184. Zhang N, Donahue J, Girshick R, Darrell T (2014) Part-based R-CNNs for fine-grained category detection. *Lect Notes Comput Sci (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)* 8689:834–849. [https://doi.org/10.1007/978-3-319-10590-1\\_54](https://doi.org/10.1007/978-3-319-10590-1_54)
185. Nguyen DT, Li W, Ogunbona PO (2016) Human detection from images and videos: a survey. *Pattern Recognit* 51:148–175. <https://doi.org/10.1016/J.PATCOG.2015.08.027>
186. Lin TY, Maire M, Belongie S, Hays J, Perona P, Ramanan D, Dollár P, Zitnick CL (2014) Microsoft COCO: common objects in context. *Lect Notes Comput Sci (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)* 8693:740–755. [https://doi.org/10.1007/978-3-319-10602-1\\_48](https://doi.org/10.1007/978-3-319-10602-1_48)
187. Goodfellow IJ, Shlens J, Szegedy C (2014) Explaining and harnessing adversarial examples. In: 3rd international conference on learning representations, ICLR 2015 - conference track proceedings
188. Selvaraju RR, Cogswell M, Das A, Vedantam R, Parikh D, Batra D (2020) Grad-CAM: visual explanations from deep networks via gradient-based localization. *Int J Comput Vis* 128:336–359. <https://doi.org/10.1007/S11263-019-01228-7/FIGURES/21>
189. Ribeiro MT, Singh S, Guestrin C (2016) Why should I trust you? Explaining the predictions of any classifier. In: *Proceedings of the ACM SIGKDD international conference on knowledge discovery and data mining*, pp 1135–1144. [https://doi.org/10.1145/2939672.2939778/SUPPL\\_FILE/KDD2016\\_RIBEIRO\\_ANY\\_CLASSIFIER\\_01-ACM.MP4](https://doi.org/10.1145/2939672.2939778/SUPPL_FILE/KDD2016_RIBEIRO_ANY_CLASSIFIER_01-ACM.MP4)
190. Lundberg SM, Allen PG, Lee S-I (2017) A unified approach to interpreting model predictions. *Adv Neural Inf Process Syst* 30
191. Esteva A, Kuprel B, Novoa RA, Ko J, Swetter SM, Blau HM, Thrun S (2017) Dermatologist-level classification of skin cancer with deep neural networks. *Nature* 542(7639):115–118
192. Sundararajan M, Taly A, Yan Q (2017) Axiomatic attribution for deep networks. In: *International conference on machine learning*, pp 3319–3328
193. Goodman B, Flaxman S (2017) European Union regulations on algorithmic decision-making and a “right to explanation.” *AI Mag* 38:50–57. <https://doi.org/10.1609/AIMAG.V38I3.2741>
194. Gilpin LH, Bau D, Yuan BZ, Bajwa A, Specter M, Kagal L (2018) Explaining explanations: an overview of interpretability of machine learning. In: *Proceedings - 2018 IEEE 5th international conference on data science and advanced analytics, DSAA 2018*, pp 80–89. <https://doi.org/10.1109/DSAA.2018.00018>
195. Doshi-Velez F, Kim B (2017) Towards a rigorous science of interpretable machine learning
196. Lee, C.Y., Gallagher, P.W. and Tu, Z., 2016, May. Generalizing pooling functions in convolutional neural networks: Mixed, gated, and tree. In *Artificial intelligence and statistics* (pp. 464–472). PMLR.
197. Krizhevsky, A., Sutskever, I. and Hinton, G.E., 2012. Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems*, 25.

**Publisher's Note** Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

## Authors and Affiliations

**Md. Himel Reza<sup>1</sup> · Md. Noman Biswas Sibly<sup>1</sup> · Shaikh Golam Rabbani<sup>1</sup> · Shafayetul Huda Sadi<sup>1</sup> · Md. Faysal Ahamed<sup>1</sup> · Fariya Bintay Shafi<sup>1</sup> · Rusab Sarmun<sup>2</sup> · Muhammad E. H. Chowdhury<sup>2</sup> **

✉ Muhammad E. H. Chowdhury  
mchowdhury@qu.edu.qa

Md. Himel Reza  
himelreza7082@gmail.com

Md. Noman Biswas Sibly  
mdnomanbiswasibly@gmail.com

Shaikh Golam Rabbani  
shaikhgolamrabbani@gmail.com

Shafayetul Huda Sadi  
shafayet.sadi@gmail.com

Md. Faysal Ahamed  
faysal.ahamed@ece.ruet.ac.bd

Fariya Bintay Shafi  
1710027@student.ruet.ac.bd

Rusab Sarmun  
rusab-2016315002@eee.du.ac.bd

<sup>1</sup> Department of Electrical and Computer Engineering, Rajshahi University of Engineering and Technology, Rajshahi 6204, Bangladesh

<sup>2</sup> Department of Electrical Engineering, Qatar University, 2713 Doha, Qatar